

ΧΑΡΟΚΟΠΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΜΑΤΙΚΗΣ

Βασίλειος Γαρουφαλής: ap25009
Κωνσταντίνος Ρουμελιώτης: ap25012

Εργασία στην ενότητα Web Development

2η Εργασία | 15 -02-2026

Εισαγωγή

Το θέμα που επιλέξαμε είναι το Σύστημα Διαχείρισης Γυμναστηρίου και η εφαρμογή που προσπαθήσαμε να υλοποιήσουμε ονομάζεται Gym Manager. Έχει σκοπό να προσφέρει ένα περιβάλλον απλό στη χρήση του και ταυτόχρονα γρήγορο ως προς την εισαγωγή των στοιχείων των μελών αλλά και εγκυρότητα αυτών.

Η πλοήγηση γίνεται μέσα από μια κεντρική γραμμή στο πάνω μέρος της οθόνης, που δίνει άμεση πρόσβαση σε όλα όσα χρειάζεται ο χρήστης. Από εκεί, μπορεί κανείς να δει τη λίστα των μελών ή να κάνει μια νέα εγγραφή, καθώς και να περιηγηθεί στα διαθέσιμα μαθήματα, όπως Yoga, CrossFit κλπ, επιλέγοντας το πρόγραμμα που προτιμά και βλέποντας ποιος γυμναστής το αναλαμβάνει.

Αρχιτεκτονική

Στην αρχή φτιάξαμε στο αρχείο `tsadimas_models.py` τις κλάσεις στις οποίες στηρίζεται η εφαρμογή μας και προσθέσαμε τα γνωρίσματα για κάθε μια από αυτές. Οι κλάσεις είναι Member Instructor και GymClass. Τέλος, οι κλάσεις DBMember, DBInstructor και DBGymClass χρησιμοποιούνται για να μετατρέψουμε τα δεδομένα της βάσης σε αντικείμενα της Python, προσθέτοντας σε αυτά και το μοναδικό ID της κάθε εγγραφής.

Στην κλάση Member δημιουργήσαμε έναν περιορισμό ηλικίας: ο χρήστης θα πρέπει να είναι από 10 έως 90 χρονών. Θέλαμε η ηλικία των μελών να υπολογίζεται αυτόματα και να είναι πάντα σωστή. Έτσι, χρησιμοποιήσαμε τη βιβλιοθήκη `datetime` για να παίρνουμε την τρέχουσα χρονολογία από το σύστημα. Αν ο χρήστης δώσει έτος γέννησης που αντιστοιχεί σε ηλικία άνω των 90 ετών και κάτω των 10 τότε το σύστημα 'σηκώνει' ένα σφάλμα (`ValueError`) πριν καν προσπαθήσει να μιλήσει με τη βάση δεδομένων και στην ιστοσελίδα εμφανίζει το αντίστοιχο μήνυμα.

ap25009_ap25012_web_app > tsadimas_models.py > ...

```python
from datetime import datetime as dtime

# --- Member Models ---
class Member:
    def __init__(self, first_name, last_name, email, birth_year):
        self.first_name = first_name
        self.last_name = last_name
        self.email = email
        self.birth_year = int(birth_year)

        # --- ΠΕΡΙΟΡΙΣΜΟΣ ΗΛΙΚΙΑΣ (10 - 90 ετών) ---
        current_year = dtime.now().year
        age = current_year - self.birth_year

        if age < 10 or age > 90:

            raise ValueError(f"Η ηλικία πρέπει να είναι μεταξύ 10 και 90 ετών. (Ηλικία: {age})

    def __str__(self):
        return f'Member: {self.first_name} {self.last_name}'

    @property
    def age(self):
        return dtime.now().year - self.birth_year # Υπολογισμός ηλικίας


class DBMember(Member):
    def __init__(self, id, first_name, last_name, email, birth_year):
        self.id = id
        self.first_name = first_name
        self.last_name = last_name
        self.email = email
        self.birth_year = int(birth_year)
```

ap25009_ap25012_web_app > tsadimas_models.py > ...

```python
# --- Instructor Models ---
class Instructor:
    def __init__(self, name, specialty):
        self.name = name
        self.specialty = specialty

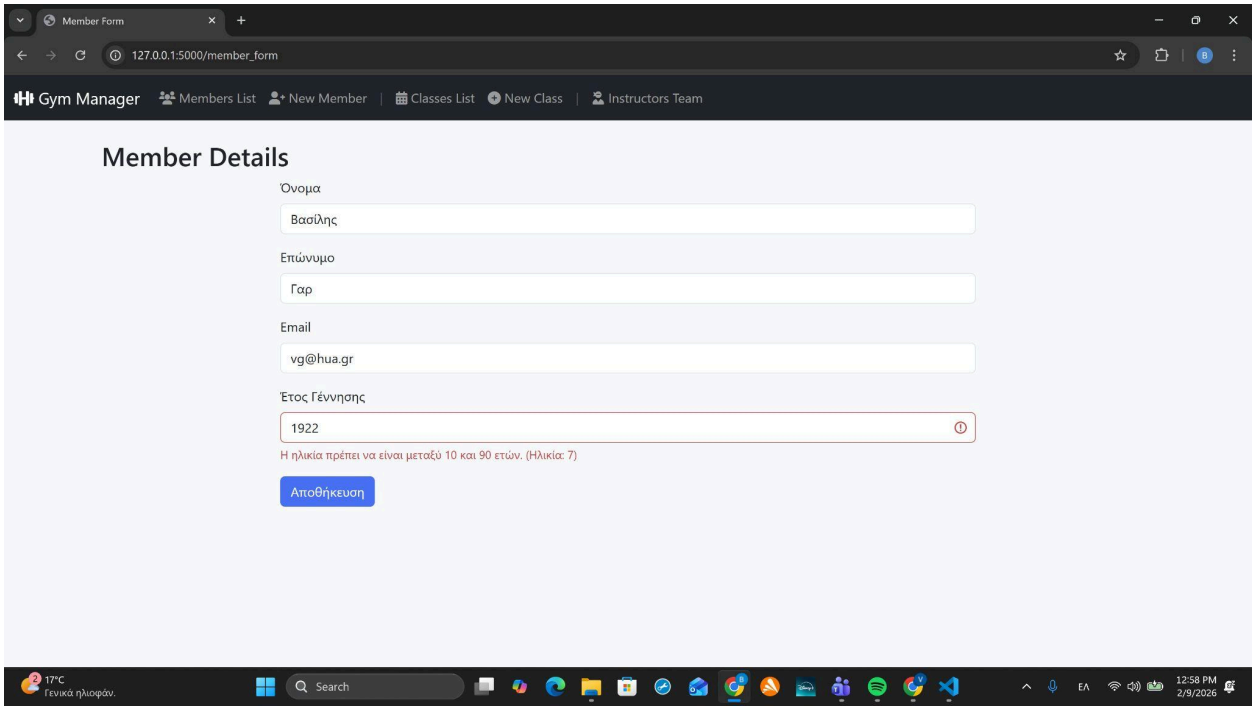
    def __str__(self):
        return f'Instructor: {self.name} ({self.specialty})'

class DBInstructor(Instructor):
    def __init__(self, id, name, specialty):
        self.id = id
        self.name = name
        self.specialty = specialty


# --- Class (GymClass) Models ---
class GymClass:
    def __init__(self, name, schedule_day, schedule_time, instructor_id):
        self.name = name
        self.schedule_day = schedule_day
        self.schedule_time = schedule_time
        self.instructor_id = instructor_id

    def __str__(self):
        return f'Class: {self.name} on {self.schedule_day}'

class DBGymClass(GymClass):
    def __init__(self, id, name, schedule_day, schedule_time, instructor_id, instructor_name=None):
        self.id = id
        self.name = name
        self.schedule_day = schedule_day
        self.schedule_time = schedule_time
        self.instructor_id = instructor_id
        # Extra πεδίο για να εμφανίζουμε το όνομα του γυμναστή στα lists
        self.instructor_name = instructor_name
```



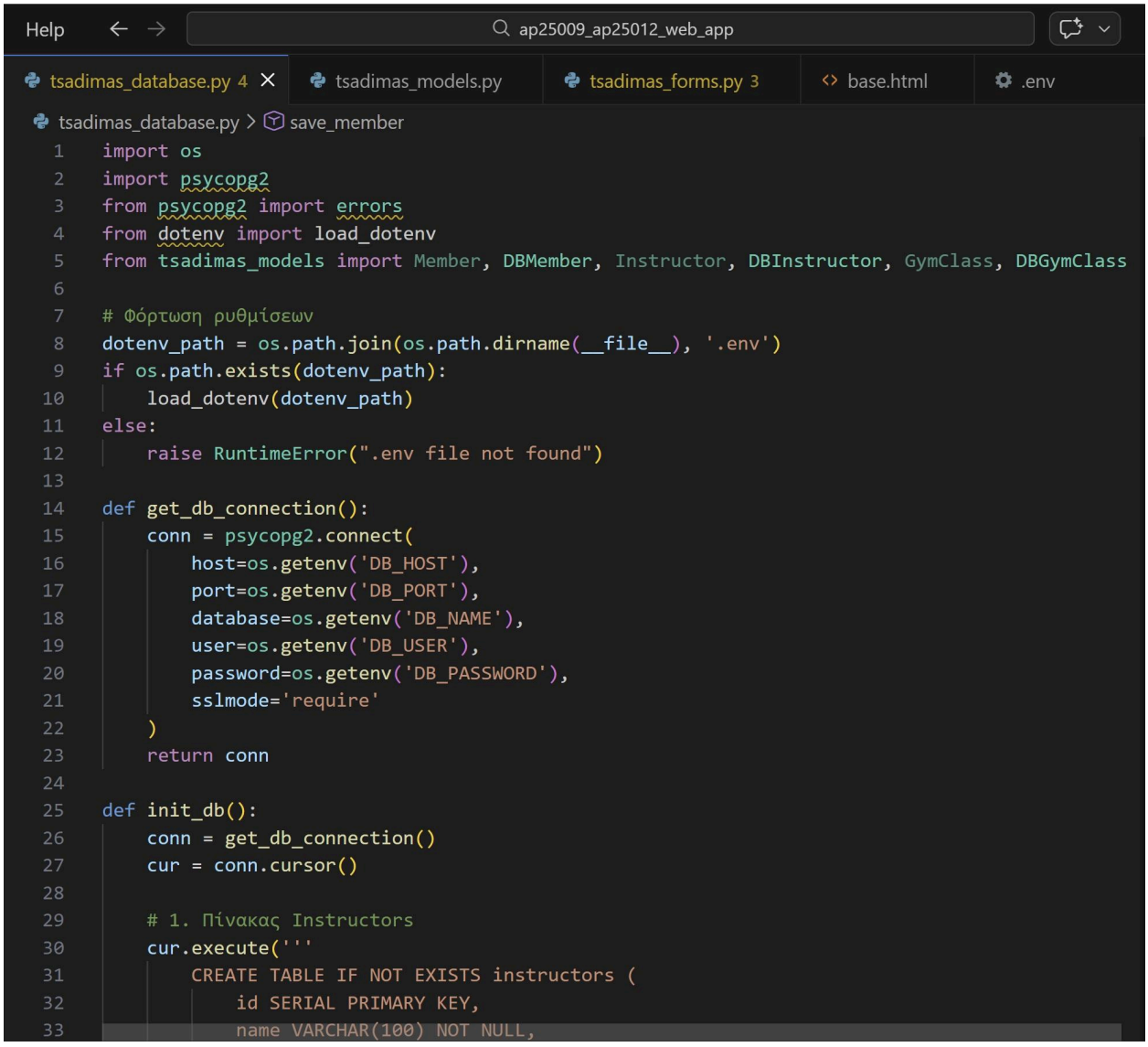
Δημιουργία πινάκων και σύνδεση της βάσης δεδομένων

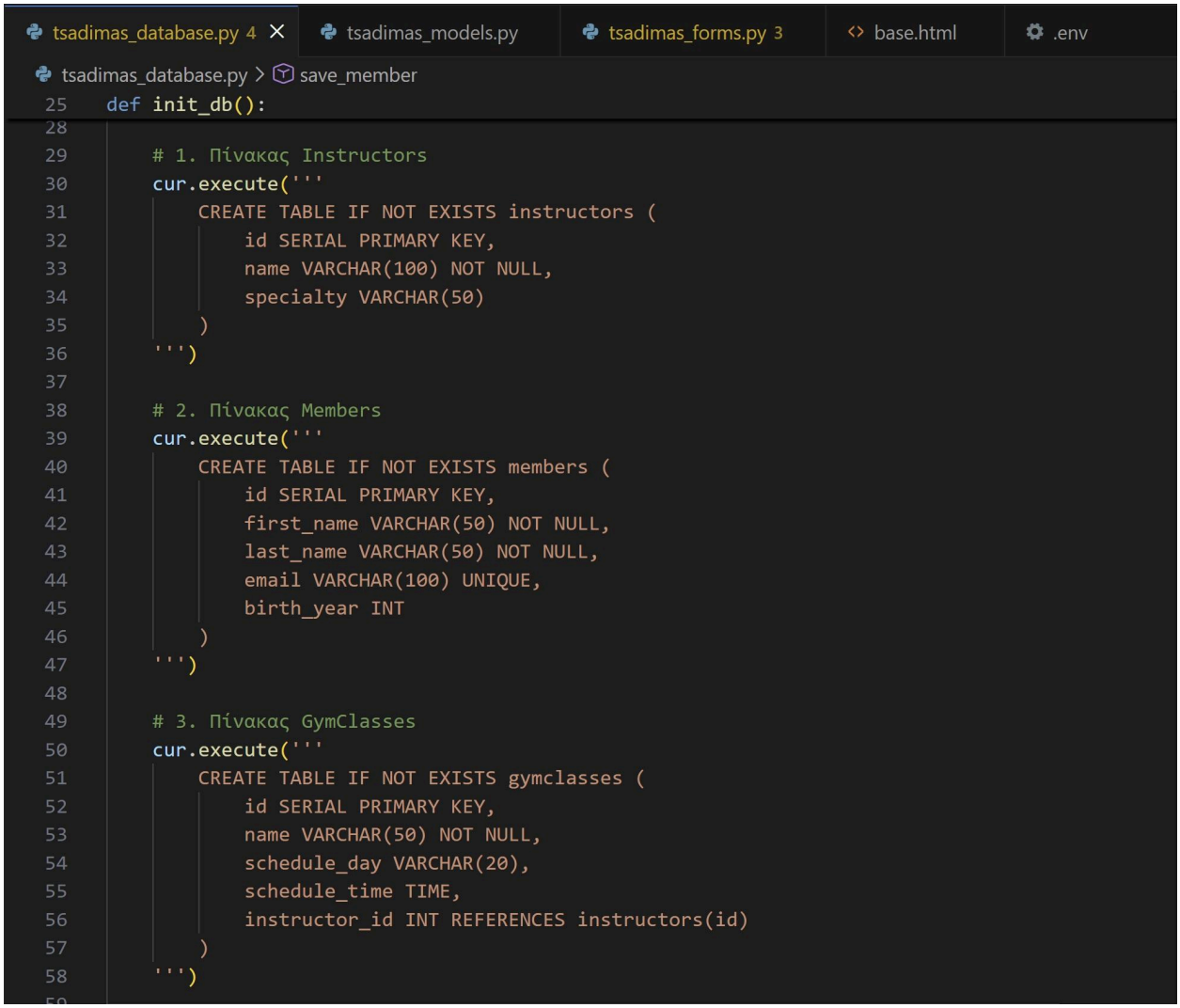
Ξεκινήσαμε με τη δημιουργία της σύνδεσης μεταξύ της εφαρμογής και της βάσης δεδομένων, με την προϋπόθεση της ασφάλεια των δεδομένων μας. Για τη διαχείριση των ευαίσθητων πληροφοριών, όπως οι κωδικοί πρόσβασης και το όνομα της βάσης, χρησιμοποιήσαμε τη βιβλιοθήκη `dotenv`. Με αυτήν, η εφαρμογή μπορεί να διαβάζει τις ευαίσθητες πληροφορίες από ένα αρχείο .env που δημιουργήσαμε μετατρέποντας οτι χρειαζομασταν σε μεταβλητες, διατηρώντας τον κώδικα «καθαρό» και προστατευμένο (όπως αναφέρθηκε και στο μάθημα).

Χρησιμοποιήσαμε την βιβλιοθήκη `psycopg2`, η οποία επιτρέπει στην python να στέλνει εντολές SQL απευθείας στη βάση. Στη συνάρτηση `get_db_connection`, βάλαμε `sslmode='require'`, ώστε κάθε ανταλλαγή πληροφοριών να είναι κρυπτογραφημένη και ασφαλής.

Στην συνέχεια, προχωρήσαμε στον καθορισμό της δομής των δεδομένων μέσω της συνάρτησης `init_db`. Σε αυτό το στάδιο, εφαρμόσαμε περιορισμούς για να αποφύγουμε λανθασμένες καταχωρήσεις. Αυτό έγινε με την ένταξη των παρακάτω περιορισμών.

- Περιορισμός **NOT NULL** στα πεδία του ονόματος και του επωνύμου στον πίνακα `members`, ώστε να μην αποθηκεύονται μέλη με ελλιπή στοιχεία.
- Περιορισμός του email ως **UNIQUE**, δημιουργώντας μια οριστική δικλείδα ασφαλείας κατά των διπλοεγγραφών.
- Για τον πίνακα `gymclasses`, επιλέξαμε τον περιορισμό **TIME** για την ώρα των μαθημάτων και δημιουργήσαμε μια σχέση **FOREIGN KEY** με τον πίνακα των γυμναστών, εξασφαλίζοντας ότι κάθε πρόγραμμα συνδέεται υποχρεωτικά με μόνο έναν instructor.





Για να μη ξεκινάει η εφαρμογή άδεια, φροντίσαμε να υπάρχουν έτοιμα δεδομένα.

Στο αρχείο `tsadimas_database.py`, ο κώδικας κάνει τον έλεγχο αν υπάρχουν γυμναστες στη βάση και αν η βάση είναι άδεια. Δημιουργεί αυτόματα τους τρεις βασικούς γυμναστές (Γιάννης, Μανώλης, Πέτρος).Έτσι, ο χρήστης μπορεί να “φτιάξει” μαθήματα αμέσως, χωρίς να χρειάζεται να καταχωρεί γυμναστές από την αρχή.

```python

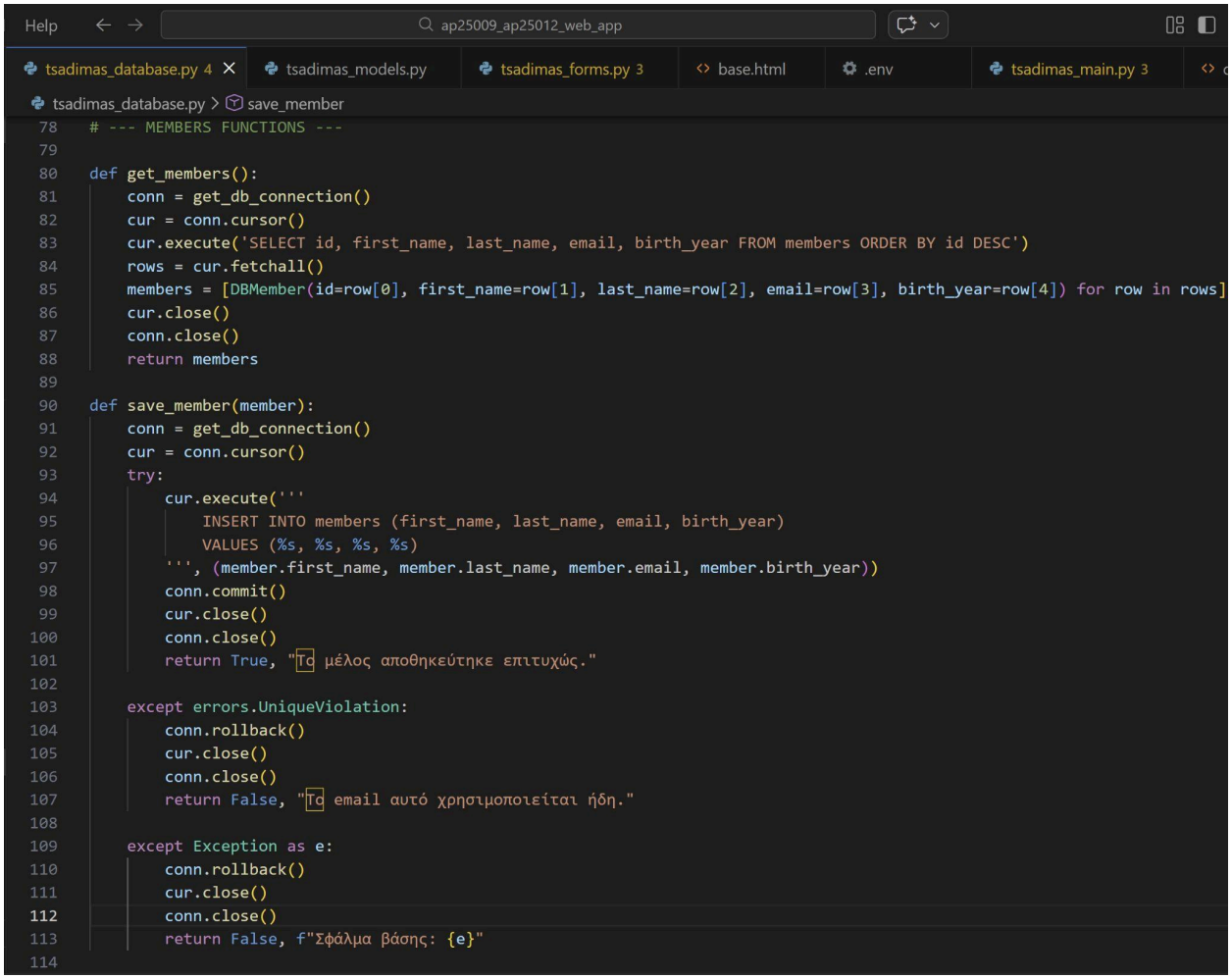
    # --- Εισαγωγή Γυμναστών ---
    cur.execute('SELECT COUNT(*) FROM instructors')
    count = cur.fetchone()[0]

    if count == 0:
        instructors_data = [
            ('Γιάννης', 'Body building', 'TRX'),
            ('Μανώλης', 'Pilates', 'Yoga'),
            ('Πέτρος', 'Crossfit')
        ]
        cur.executemany('INSERT INTO instructors (name, specialty) VALUES (%s, %s)', instructors_data)
        print("Προστέθηκαν οι γυμναστές: Γιάννης, Μανώλης, Πέτρος")

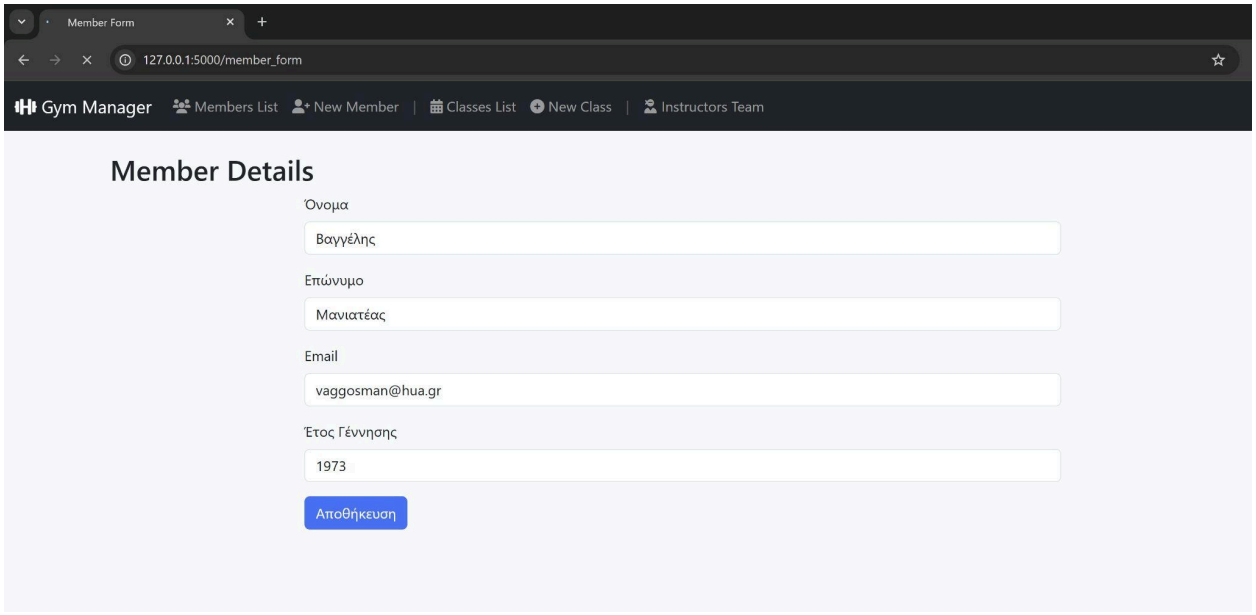
    conn.commit()
    cur.close()
    conn.close()
```

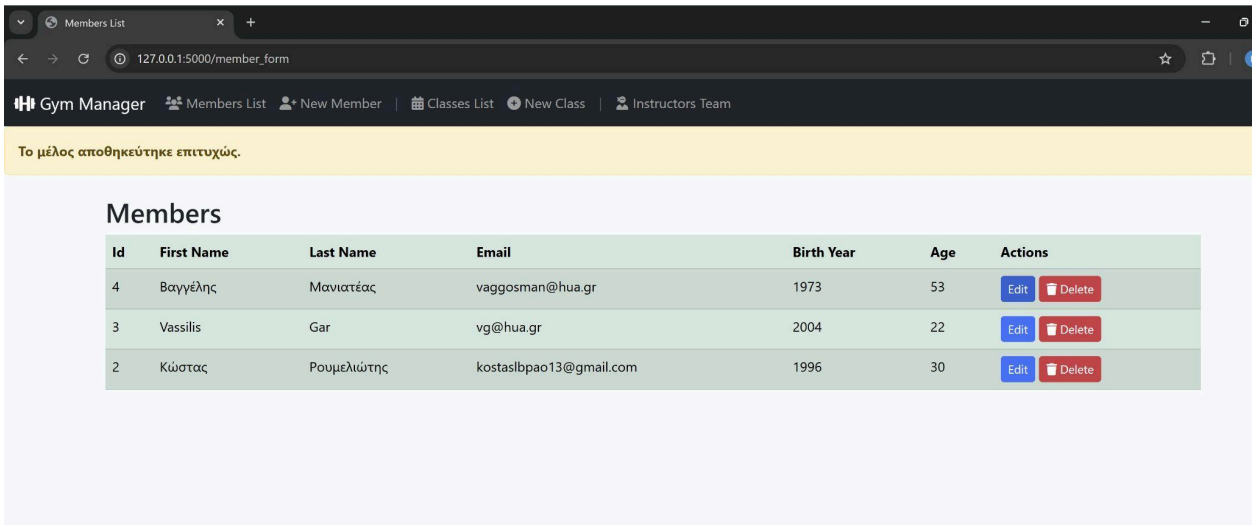
Φτιάξαμε την συνάρτηση `get_members` η οποία φέρνει όλα τα μέλη από την λίστα με τα στοιχεία τους.

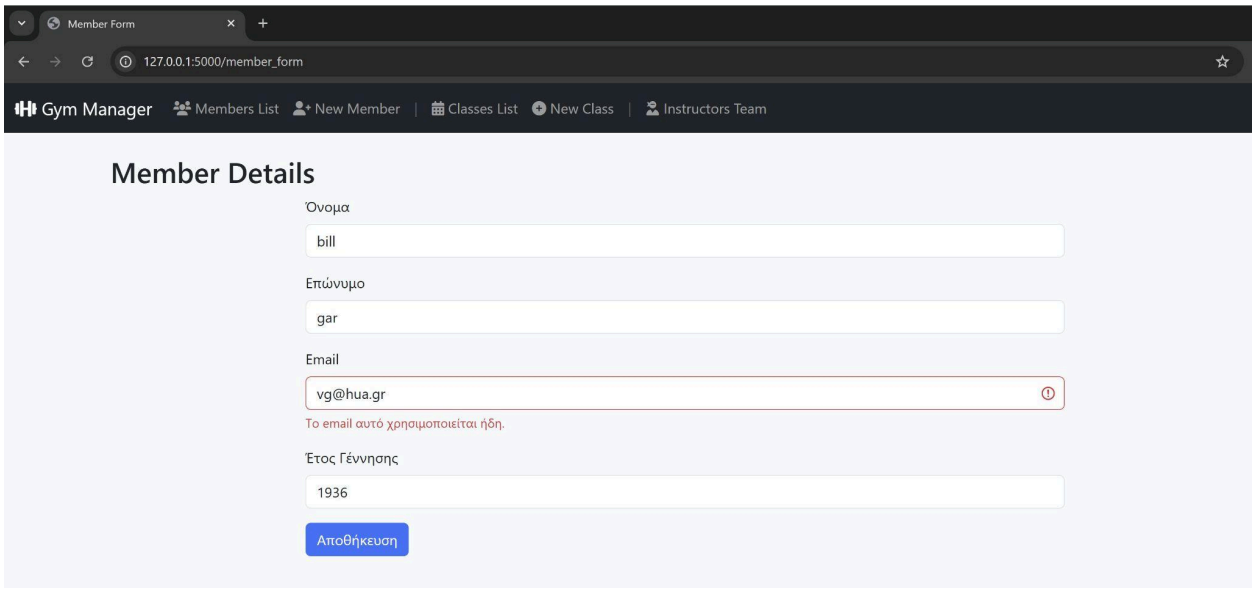
Στην συνέχεια φτιαξαμε την συνάρτηση `save_member`. Στην συγκεκριμένη συνάρτηση δημιουργούνται νέα μέλη, γίνεται έλεγχος για την μοναδικότητα των email (errors.UniqueViolation) και με το τελευταίο exception γίνεται έλεγχος για την αποφυγή σφαλμάτων. Όλα αυτά γίνονται σε ένα μπλοκ ελέγχου **try...except**. Έτσι, αν ένας χρήστης προσπαθήσει να χρησιμοποιήσει ένα ήδη καταχωρημένο email, η εφαρμογή δεν “σκάει”, αλλά εμφανίζει το ενημερωτικό μήνυμα: «Το email αυτό χρησιμοποιείται ήδη».



Στην ιστοσελίδα φαίνεται:

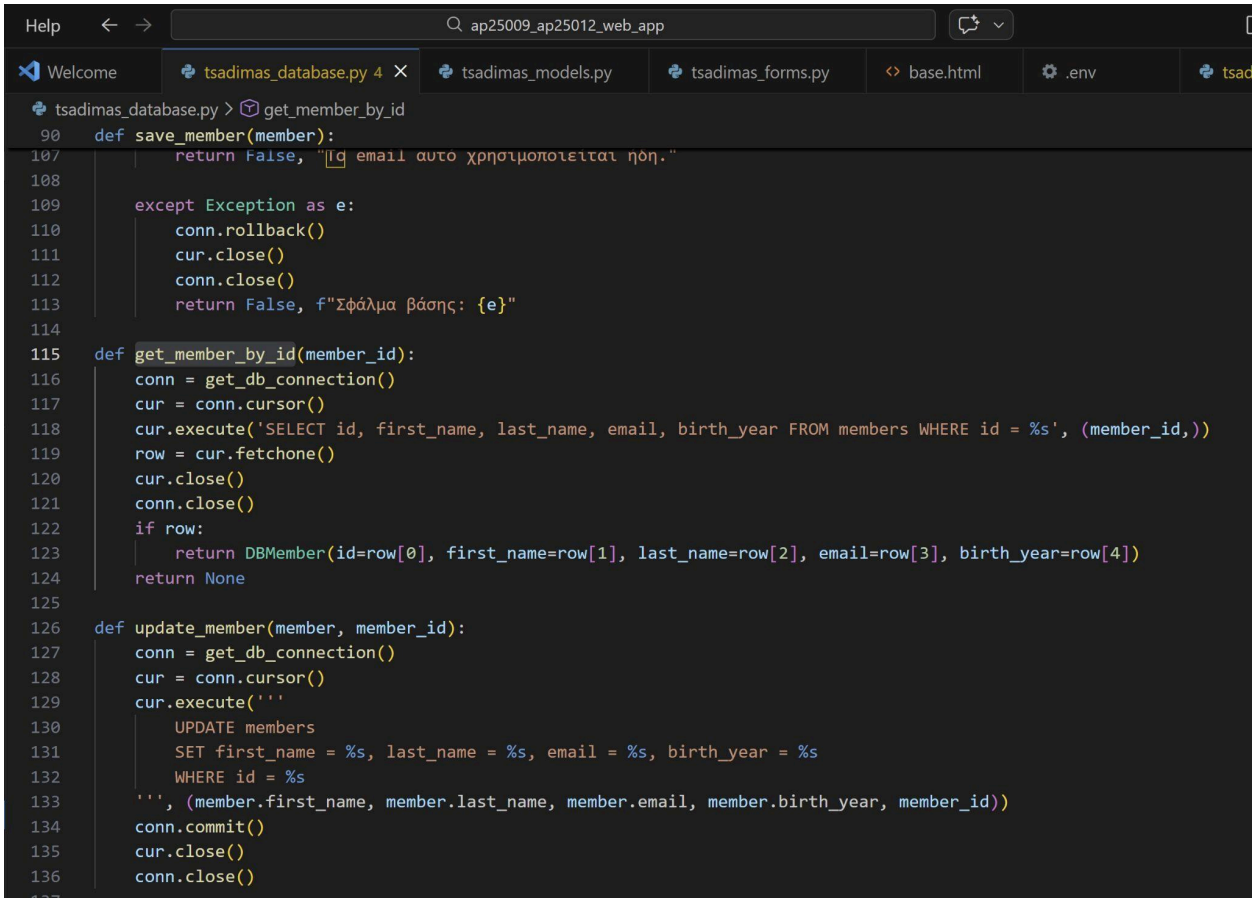

Member Form
127.0.0.1:5000/member_form
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
Member Details
Όνομα
Βαγγέλης
Επώνυμο
Μανιατέας
Email
vaggosman@hua.gr
Έτος Γέννησης
1973
Αποθήκευση

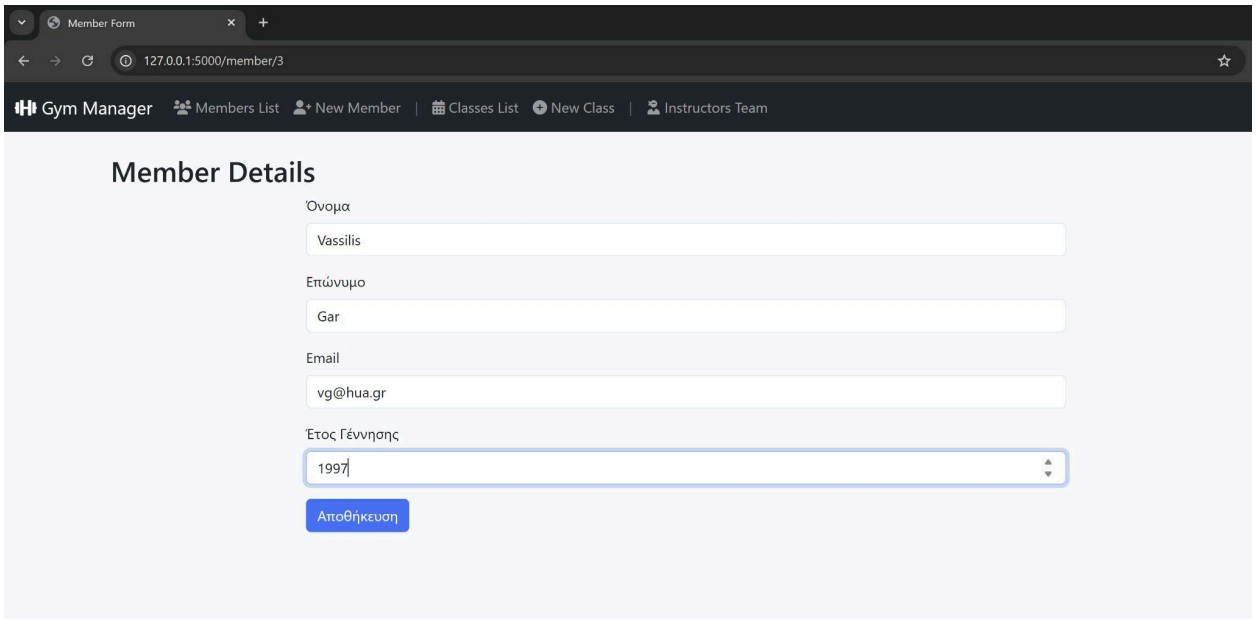

Members List
127.0.0.1:5000/member_form
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
Το μέλος αποθηκεύτηκε επιτυχώς.
Members
Id
First Name
Last Name
Email
Birth Year
Age
Actions
4
Βαγγέλης
Μανιατέας
vaggosman@hua.gr
1973
53
Edit
Delete
3
Vassilis
Gar
vg@hua.gr
2004
22
Edit
Delete
2
Κώστας
Ρουμελιώτης
kostaslbpao13@gmail.com
1996
30
Edit
Delete

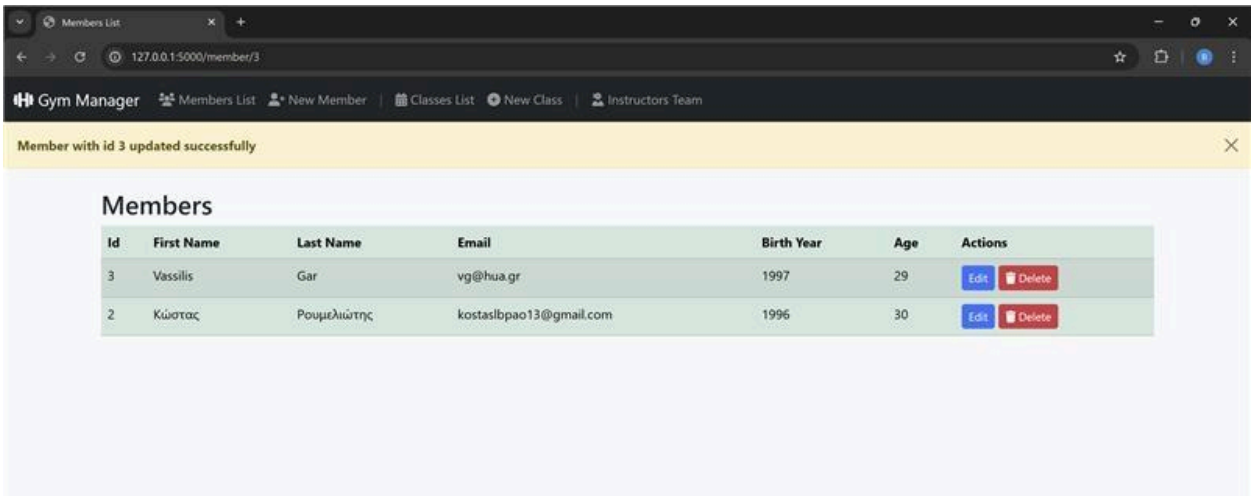


Η `get_member_by_id` βρίσκει ένα συγκεκριμένο μέλος που θέλουμε να επεξεργαστούμε.

Με την συνάρτηση `update_member` μπορούμε να αλλάξουμε τα στοιχεία ενός μέλους Αυτή η λειτουργία χρειάζεται δύο βασικές πληροφορίες: την **ταυτότητα** του μέλους που φαίνεται από το `member_id`, για να ξέρει το σύστημα σε ποιο άτομο αναφερόμαστε, και τα **νέα στοιχεία** που θέλουμε να αποθηκεύσουμε.

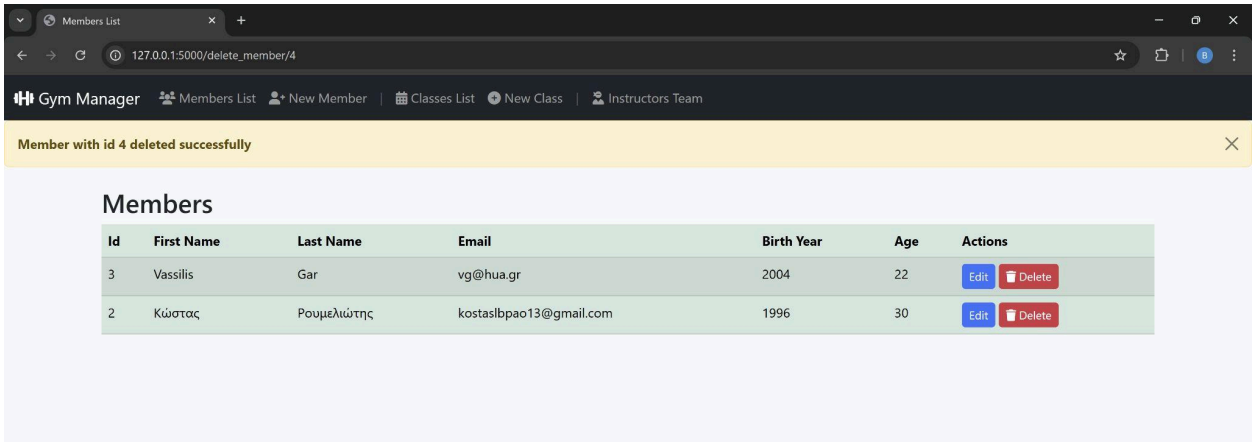

Help
ap25009_ap25012_web_app
Welcome
tsadimas_database.py 4
tsadimas_models.py
tsadimas_forms.py
base.html
.env
tsadimas_database.py > get_member_by_id
90 def save_member(member):
107 return False, "Το email αυτό χρησιμοποιείται ήδη."
108
109 except Exception as e:
110 conn.rollback()
111 cur.close()
112 conn.close()
113 return False, f"Σφάλμα βάσης: {e}"
114
115 def get_member_by_id(member_id):
116 conn = get_db_connection()
117 cur = conn.cursor()
118 cur.execute('SELECT id, first_name, last_name, email, birth_year FROM members WHERE id = %s', (member_id,))
119 row = cur.fetchone()
120 cur.close()
121 conn.close()
122 if row:
123 return DBMember(id=row[0], first_name=row[1], last_name=row[2], email=row[3], birth_year=row[4])
124 return None
125
126 def update_member(member, member_id):
127 conn = get_db_connection()
128 cur = conn.cursor()
129 cur.execute('''
130 UPDATE members
131 SET first_name = %s, last_name = %s, email = %s, birth_year = %s
132 WHERE id = %s
133 ''', (member.first_name, member.last_name, member.email, member.birth_year, member_id))
134 conn.commit()
135 cur.close()
136 conn.close()


Member Form
127.0.0.1:5000/member/3
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
Member Details
Όνομα
Vassilis
Επώνυμο
Gar
Email
vg@hua.gr
Έτος Γέννησης
1997
Αποθήκευση



Με την συνάρτηση `delete_member` μπορούμε να διαγράψουμε το μέλος, οπότε διαγράφονται και όλα τα στοιχεία του.

```python
def delete_member(member_id):
    conn = get_db_connection()
    cur = conn.cursor()
    # Βήμα 1: Έλεγχος αν υπάρχει
    cur.execute('SELECT * FROM members WHERE id = %s', (member_id,))
    row = cur.fetchone()
    # Αν δεν υπάρχει, επιστροφή False
    if row is None:
        cur.close()
        conn.close()
        return False
    # Βήμα 2: Διαγραφή
    cur.execute('DELETE FROM members WHERE id = %s', (member_id,))
    conn.commit()
    cur.close()
    conn.close()
    return True
```



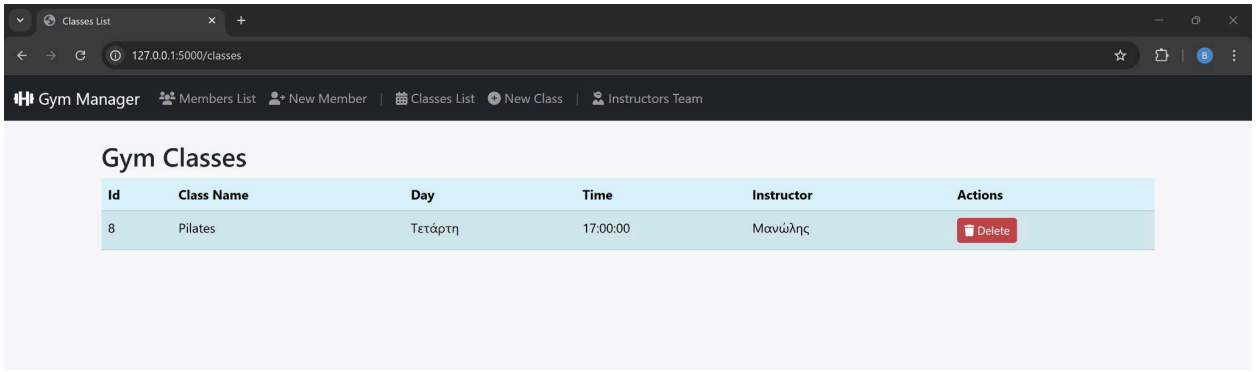
Τέλος, κατά τη διαδικασία της διαγραφής, διαπιστώσαμε ότι η προηγούμενη ανάκτηση των δεδομένων μέσω της `get_gym_classes` είναι απαραίτητη, ώστε να επιβεβαιώνεται η ύπαρξη ενός στοιχείου πριν την οριστική αφαίρεσή του από το σύστημα.

Για τη διαχείριση των μαθημάτων και των γυμναστών ακολουθούμε την ίδια λογική (προσθήκη, διαγραφή, αποθήκευση) που χρησιμοποιούμε και στα μέλη. Η μόνη διαφορά εντοπίζεται στη συνάρτηση `get_gym_classes`, η οποία εμφανίζει τη λίστα των μαθημάτων.

Επειδή στη βάση δεδομένων κάθε μάθημα αποθηκεύει το ID του γυμναστή, για να το λύσουμε αυτό, χρησιμοποιούμε την εντολή **JOIN**, η οποία ενώνει τον πίνακα των μαθημάτων με τον πίνακα των γυμναστών. Με αυτή την ένωση, η εφαρμογή βρίσκει αυτόματα το όνομα που αντιστοιχεί σε κάθε κωδικό, επιτρέποντας στον χρήστη να βλέπει τον γυμναστή με το αντίστοιχο μάθημα που κάνει πχ «Pilates - Μανώλης», κάνοντας το σύστημα πολύ πιο φιλικό.

```python
def get_gym_classes():
    conn = get_db_connection()
    cur = conn.cursor()
    #  Εδώ τραβάμε δεδομένα από το 'gymclasses'
    cur.execute('''
        SELECT c.id, c.name, c.schedule_day, c.schedule_time, c.instructor_id, i.name
        FROM gymclasses c
        JOIN instructors i ON c.instructor_id = i.id
    ''')
    rows = cur.fetchall()
    gymclasses = [DBGymClass(id=row[0], name=row[1], schedule_day=row[2], schedule_time=row[3], instructor_id=row[4],instructor_name=row[5]) for row in rows]
    cur.close()
    conn.close()
    return gymclasses

def delete_gym_class(class_id):
    conn = get_db_connection()
    cur = conn.cursor()
    # Στέλνουμε κατευθείαν την εντολή διαγραφής για το συγκεκριμένο μάθημα
    cur.execute('DELETE FROM gymclasses WHERE id = %s', (class_id,))
    conn.commit()
    cur.close()
    conn.close()
```



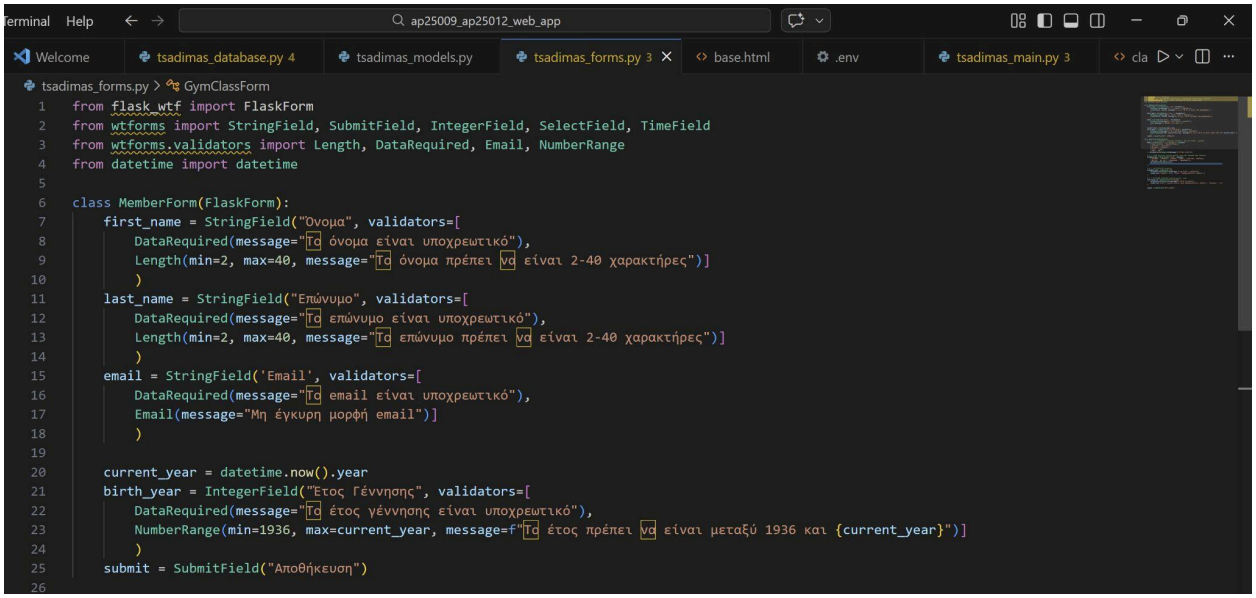
Δημιουργία των Φορμών

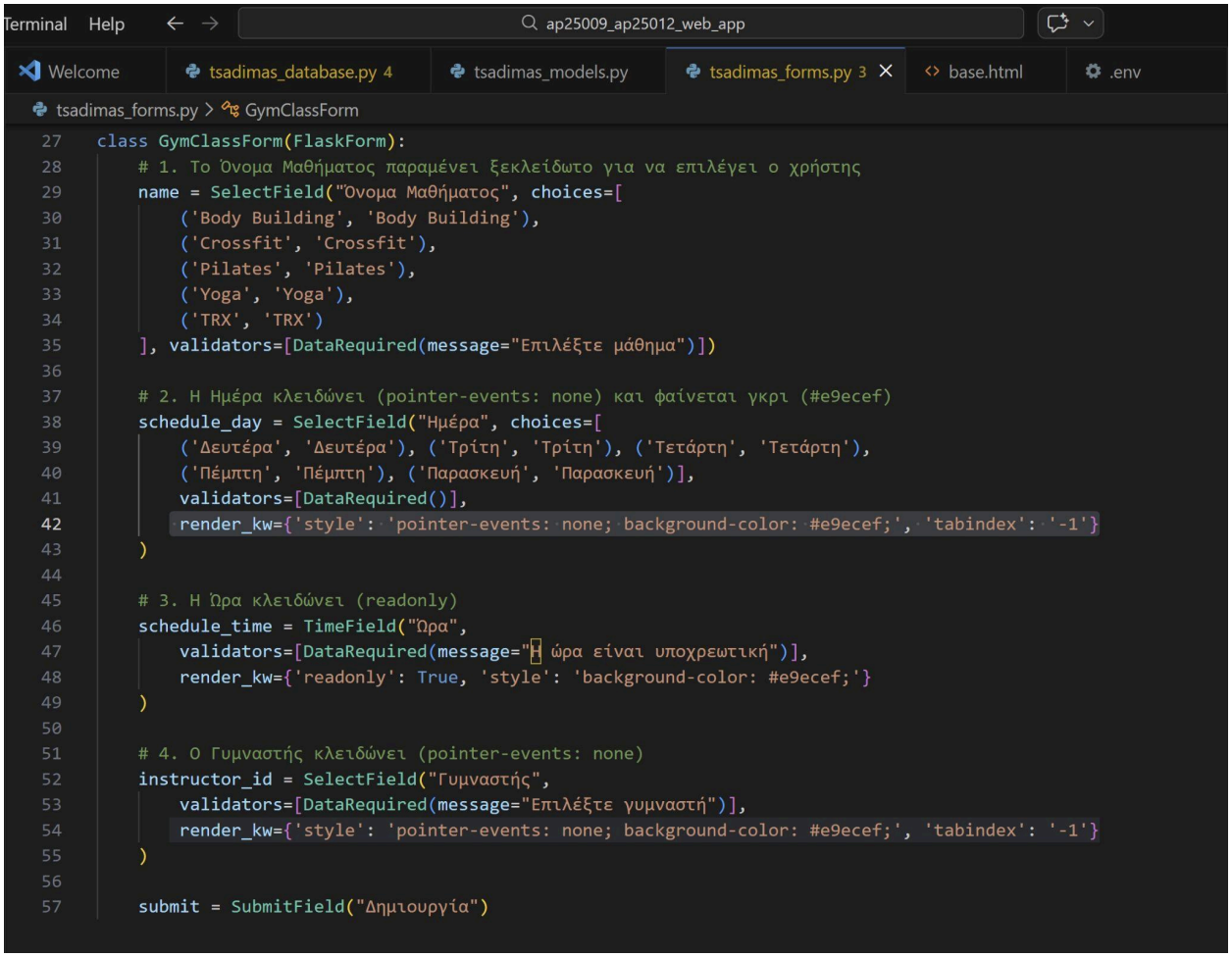
Στο αρχείο `tsadimas_forms.py`, γίνεται η υλοποίηση των φορμών μας που βασίζονται στη βιβλιοθήκη wtforms, χρησιμοποιώντας ως θεμέλιο το `FlaskForm`. Με αυτόν τον τρόπο, κάθε φόρμα αποκτά αυτόματα όλες τις απαραίτητες ρυθμίσεις ασφαλείας. Για να διευκολύνουμε τον χρήστη και να διασφαλίσουμε την ορθότητα των στοιχείων, επιλέξαμε συγκεκριμένα πεδία εισαγωγής ανάλογα με το είδος των δεδομένων. Χρησιμοποιούμε το `StringField` για κείμενο όπως το όνομα, το `IntegerField` για την ηλικία και το `SelectField` για έτοιμες επιλογές από λίστα (dropdown), ώστε να αποφεύγονται τα λάθη πληκτρολόγησης. Παράλληλα, το `TimeField` επιτρέπει την

επιλογή ώρας μέσα από το ρολόι του browser, ενώ το `SubmitField` ολοκληρώνει τη διαδικασία με το κουμπί αποστολής.

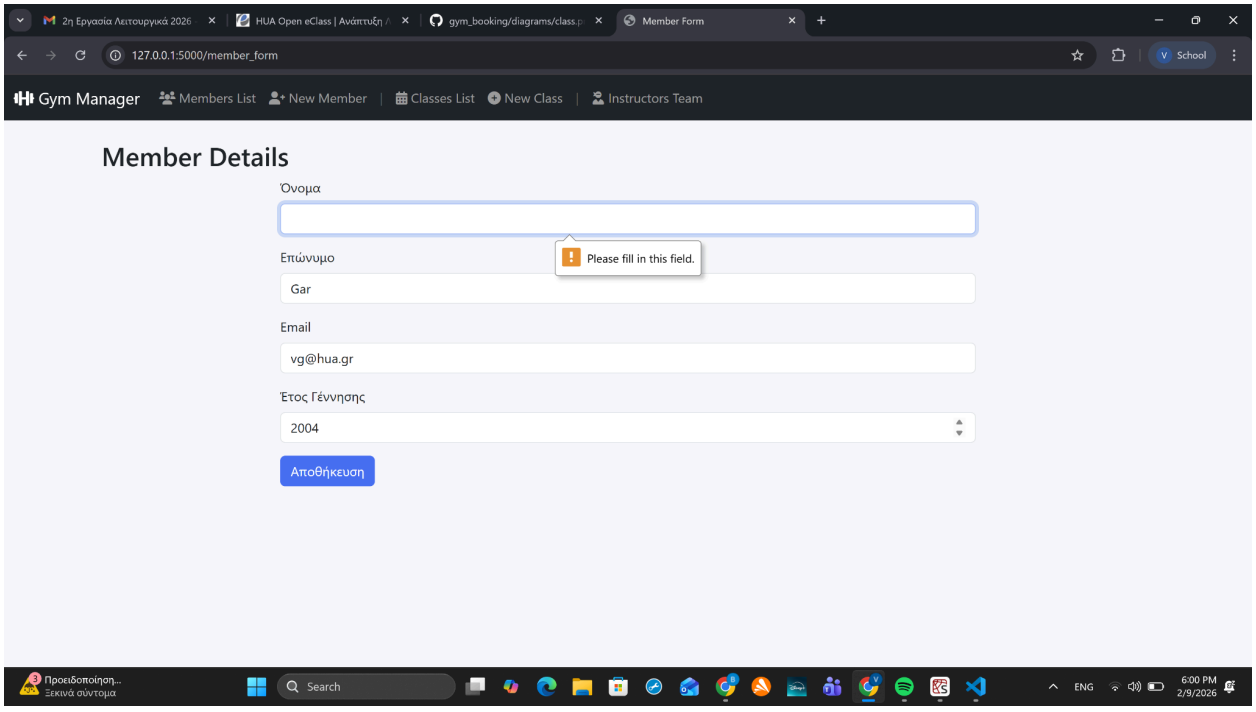
Η αξιοπιστία του συστήματος εξαρτάται από τους Validators, που αντιπροσωπεύουν τους κανόνες των δεδομένων. Αντί να κάνουμε χειροκίνητα τους ελέγχους, ορίζουμε αυστηρούς κανόνες που προλαμβάνουν τα σφάλματα πριν αυτά φτάσουν στη βάση δεδομένων και δημιουργήσουν πρόβλημα στην λειτουργία του συστήματος. Ο validator `DataRequired` διασφαλίζει ότι κανένα πεδίο δεν θα μείνει κενό, ενώ ο `Length` υποχρεώνει το κείμενο να έχει ένα λογικό μέγεθος επίσης έχουμε και πιο ειδικές περιπτώσεις, όπως είναι ο `Email validator` που καταλαβαίνει αμέσως αν ο χρήστης έγραψε κάτι τυχαίο αντί για μια σωστή ηλεκτρονική διεύθυνση και τον ενημερώνει να το διορθώσει. Τέλος, με τον `NumberRange` διασφαλίζουμε ότι σε πεδία όπως η ηλικία δεν θα καταχωρηθούν αρνητικοί αριθμοί ή εξωπραγματικές τιμές, προστατεύοντας το σύστημα από μελλοντικά error.

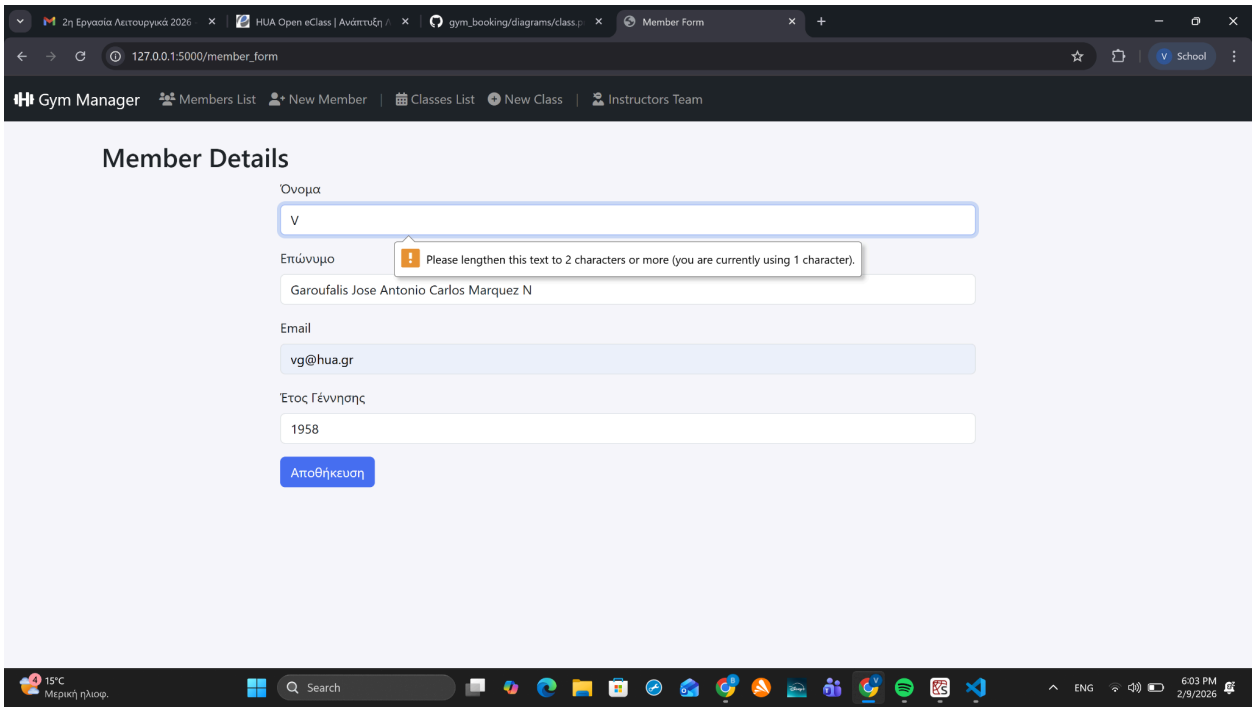
Τέλος, στη φόρμα `GymClassForm` προσθέσαμε την παράμετρο `render_kw` για να «κλείσουμε» τα πεδία της ημέρας, της ώρας και του γυμναστή. Ορίζοντάς τα ως `readonly` (και με `pointer-events: none`), επιτρέπουμε στον χρήστη να επιλέγει μόνο το μάθημα, ενώ τα υπόλοιπα στοιχεία συμπληρώνονται αυτόματα. Με αυτόν τον τρόπο, τα δεδομένα εμφανίζονται σωστά χωρίς να επιτρέπεται η χειροκίνητη τροποποίηση.

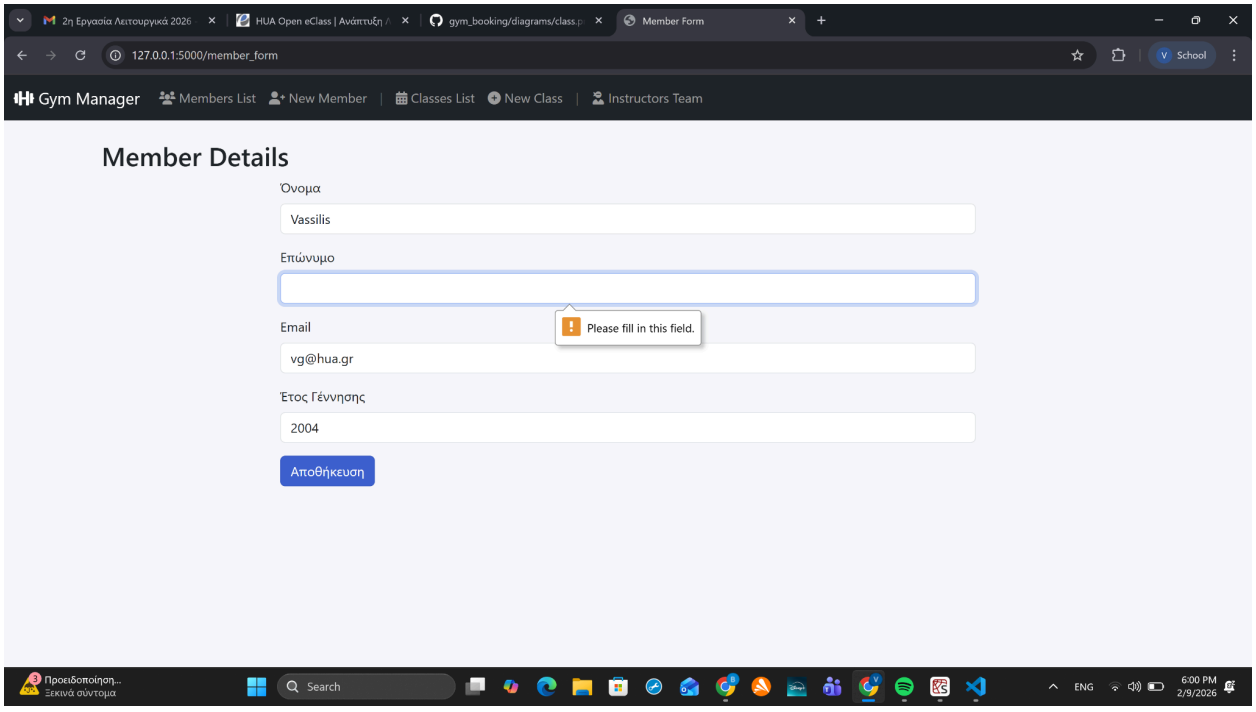

Terminal Help
ap25009_ap25012_web_app
Welcome
tsadimas_database.py 4
tsadimas_models.py
tsadimas_forms.py 3
base.html
.env
tsadimas_main.py 3
tsadimas_forms.py > GymClassForm
from flask_wtf import FlaskForm
from wtforms import StringField, SubmitField, IntegerField, SelectField, TimeField
from wtforms.validators import Length, DataRequired, Email, NumberRange
from datetime import datetime
class MemberForm(FlaskForm):
first_name = StringField("Όνομα", validators=[
DataRequired(message="Το όνομα είναι υποχρεωτικό"),
Length(min=2, max=40, message="Το όνομα πρέπει να είναι 2-40 χαρακτήρες")]
)
last_name = StringField("Επώνυμο", validators=[
DataRequired(message="Το επώνυμο είναι υποχρεωτικό"),
Length(min=2, max=40, message="Το επώνυμο πρέπει να είναι 2-40 χαρακτήρες")]
)
email = StringField('Email', validators=[
DataRequired(message="Το email είναι υποχρεωτικό"),
Email(message="Μη έγκυρη μορφή email")]
)
current_year = datetime.now().year
birth_year = IntegerField("Έτος Γέννησης", validators=[
DataRequired(message="Το έτος γέννησης είναι υποχρεωτικό"),
NumberRange(min=1936, max=current_year, message=f"Το έτος πρέπει να είναι μεταξύ 1936 και {current_year}")]
)
submit = SubmitField("Αποθήκευση")

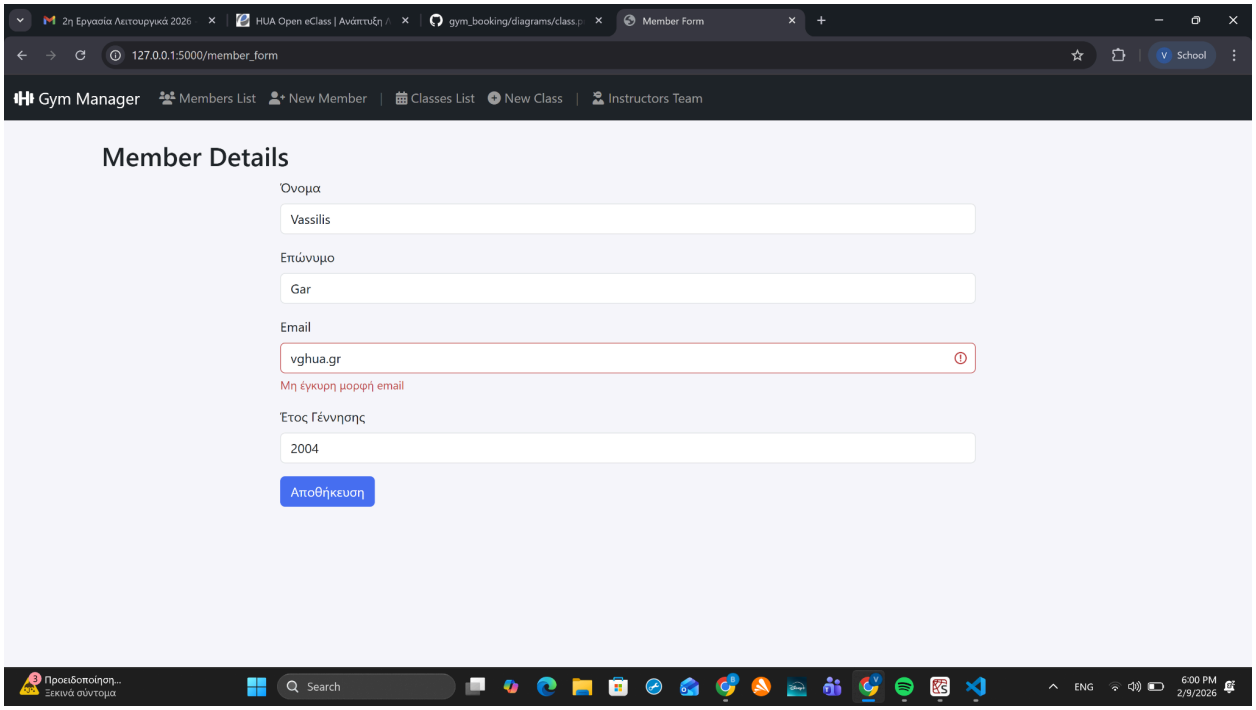

Terminal Help
ap25009_ap25012_web_app
Welcome
tsadimas_database.py 4
tsadimas_models.py
tsadimas_forms.py 3
base.html
.env
tsadimas_forms.py > GymClassForm
class GymClassForm(FlaskForm):
1. Το Όνομα Μαθήματος παραμένει ξεκλείδωτο για να επιλέγει ο χρήστης
name = SelectField("Όνομα Μαθήματος", choices=[
('Body Building', 'Body Building'),
('Crossfit', 'Crossfit'),
('Pilates', 'Pilates'),
('Yoga', 'Yoga'),
('TRX', 'TRX')
], validators=[DataRequired(message="Επιλέξτε μάθημα")])
2. Η Ημέρα κλειδώνει (pointer-events: none) και φαίνεται γκρι (#e9ecef)
schedule_day = SelectField("Ημέρα", choices=[
('Δευτέρα', 'Δευτέρα'), ('Τρίτη', 'Τρίτη'), ('Τετάρτη', 'Τετάρτη'),
('Πέμπτη', 'Πέμπτη'), ('Παρασκευή', 'Παρασκευή')],
validators=[DataRequired()],
render_kw={'style': 'pointer-events: none; background-color: #e9ecef;', 'tabindex': '-1'}
)
3. Η Ώρα κλειδώνει (readonly)
schedule_time = TimeField("Ώρα",
validators=[DataRequired(message="Η ώρα είναι υποχρεωτική")],
render_kw={'readonly': True, 'style': 'background-color: #e9ecef;'}
)
4. Ο Γυμναστής κλειδώνει (pointer-events: none)
instructor_id = SelectField("Γυμναστής",
validators=[DataRequired(message="Επιλέξτε γυμναστή")],
render_kw={'style': 'pointer-events: none; background-color: #e9ecef;', 'tabindex': '-1'}
)
submit = SubmitField("Δημιουργία")

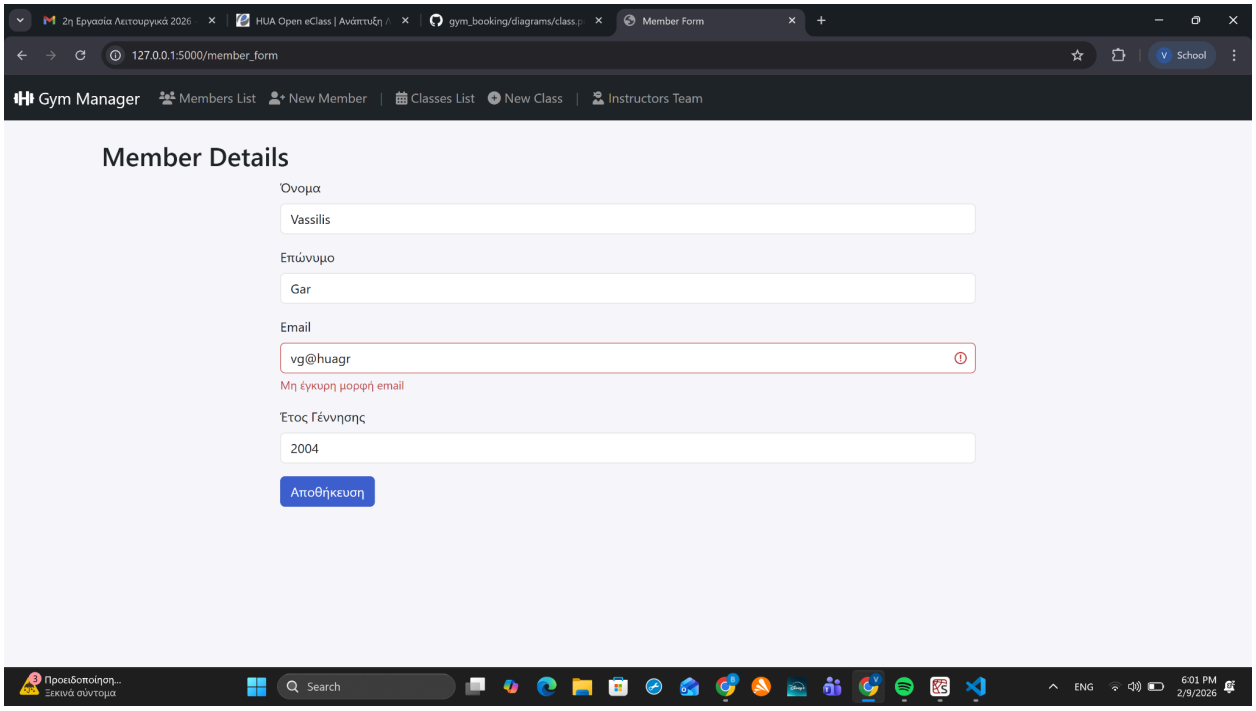
Παρακάτω φαίνονται οι περιορισμοί κατά την συμπλήρωση της φόρμας στην web εφαρμογή.

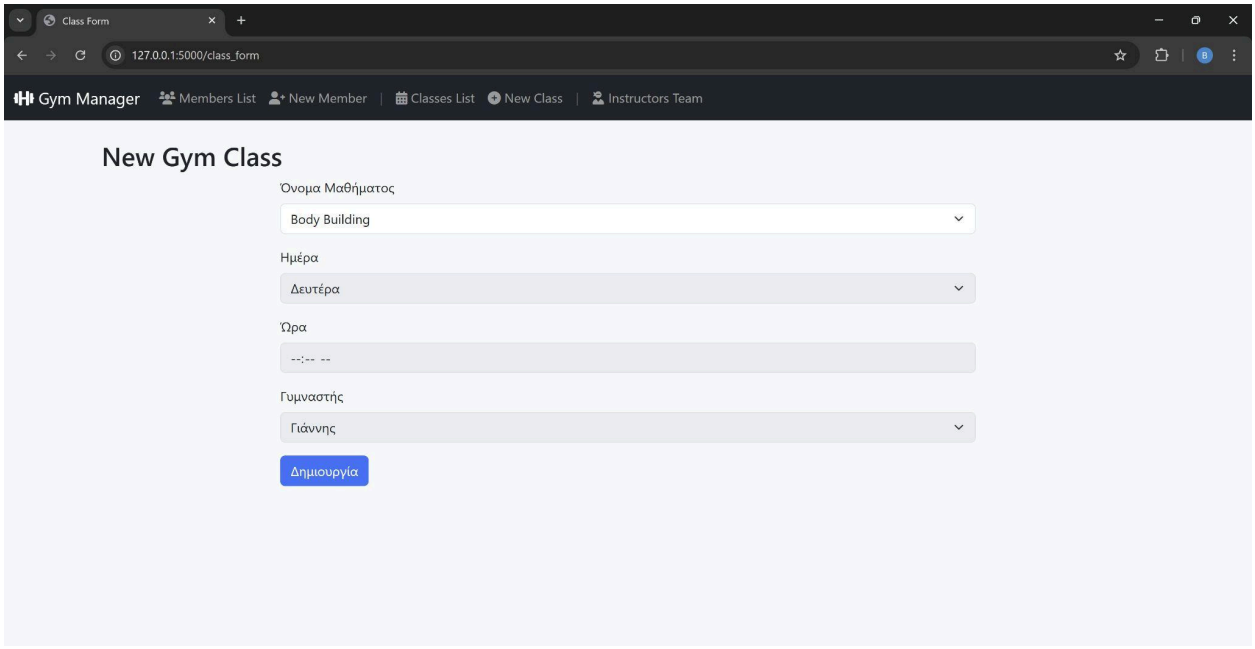


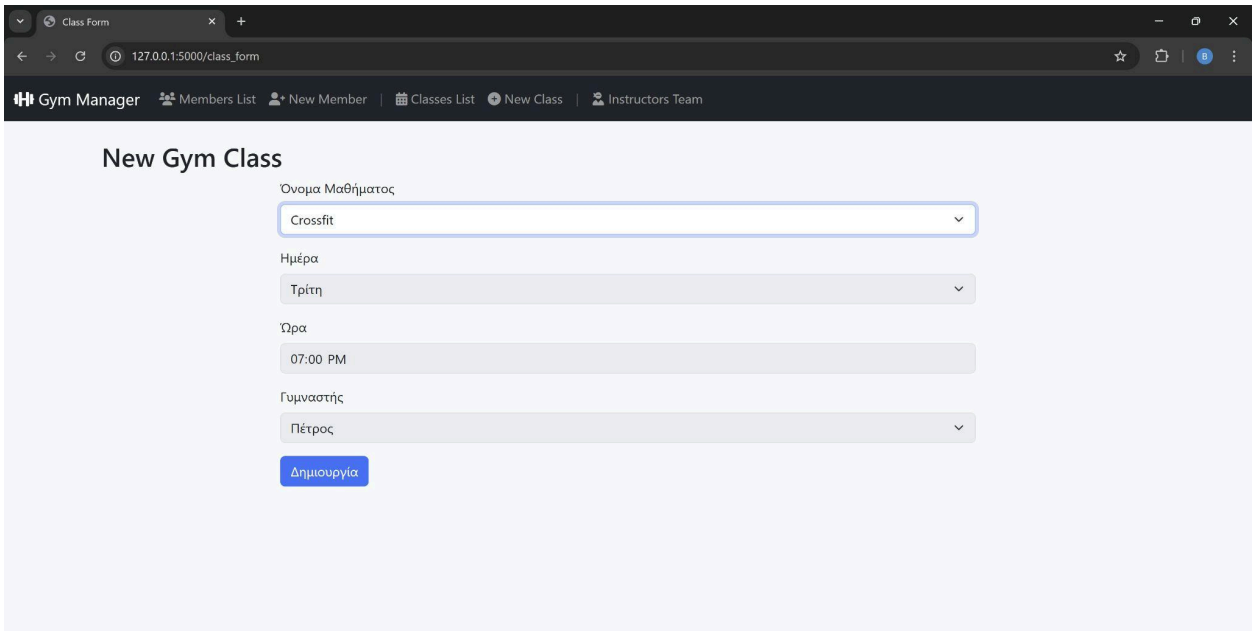

2η Εργασία Λειτουργικά 2026
HUA Open eClass | Ανάπτυξη
gym_booking/diagrams/class.p
Member Form
127.0.0.1:5000/member_form
School
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
Member Details
Όνομα
V
Please lengthen this text to 2 characters or more (you are currently using 1 character).
Επώνυμο
Garoufalis Jose Antonio Carlos Marquez N
Email
vg@hua.gr
Έτος Γέννησης
1958
Αποθήκευση
15°C
Μερική ηλιοφ.
Search
ENG
6:03 PM
2/9/2026

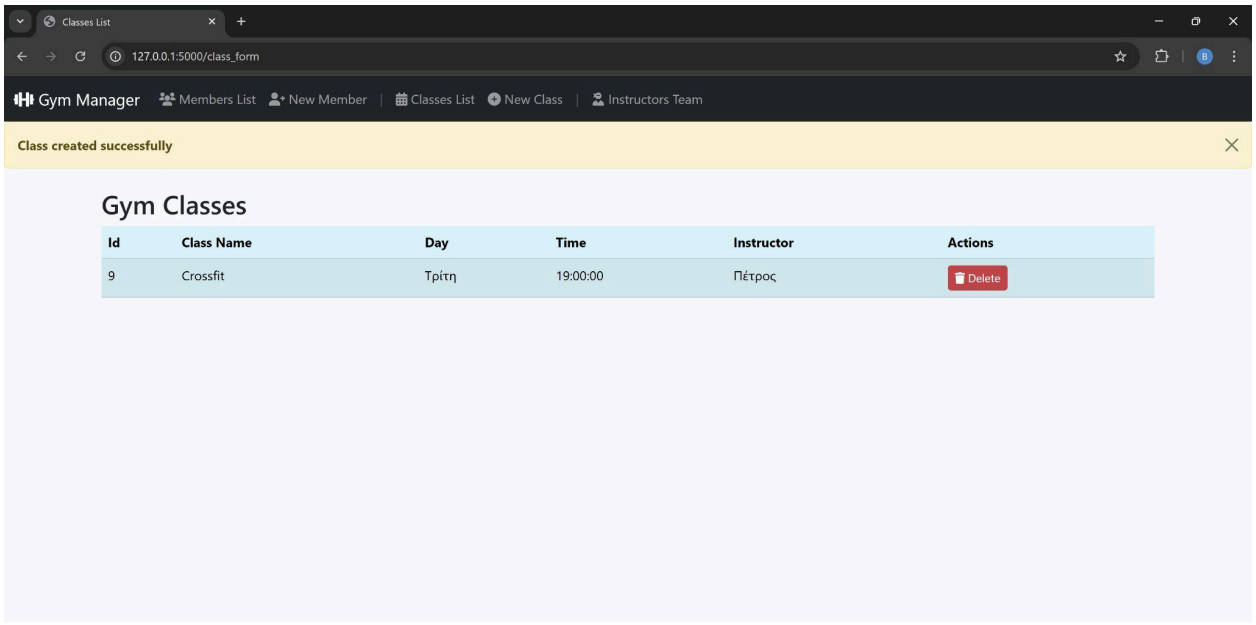

2η Εργασία Λειτουργικά 2026
HUA Open eClass | Ανάπτυξη
gym_booking/diagrams/class.p
Member Form
127.0.0.1:5000/member_form
School
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
Member Details
Όνομα
Vassilis
Επώνυμο
Please fill in this field.
Email
vg@hua.gr
Έτος Γέννησης
2004
Αποθήκευση
Προειδοποίηση...
Search
ENG
6:00 PM
2/9/2026


2η Εργασία Λειτουργικά 2026
HUA Open eClass | Ανάπτυξη
gym_booking/diagrams/class.p
Member Form
127.0.0.1:5000/member_form
School
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
Member Details
Όνομα
Vassilis
Επώνυμο
Gar
Email
vghua.gr
Μη έγκυρη μορφή email
Έτος Γέννησης
2004
Αποθήκευση
Προειδοποίηση...
Σεκινά σύντομα
Search
ENG
6:00 PM
2/9/2026


2η Εργασία Λειτουργικά 2026
HUA Open eClass | Ανάπτυξη
gym_booking/diagrams/class.p
Member Form
127.0.0.1:5000/member_form
School
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
Member Details
Όνομα
Vassilis
Επώνυμο
Gar
Email
vg@huagr
Μη έγκυρη μορφή email
Έτος Γέννησης
2004
Αποθήκευση
Προειδοποίηση...
Σεκινά σύντομα
Search
ENG
6:01 PM
2/9/2026


Class Form
127.0.0.1:5000/class_form
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
New Gym Class
Όνομα Μαθήματος
Body Building
Ημέρα
Δευτέρα
Ώρα
--:-- --
Γυμναστής
Γιάννης
Δημιουργία

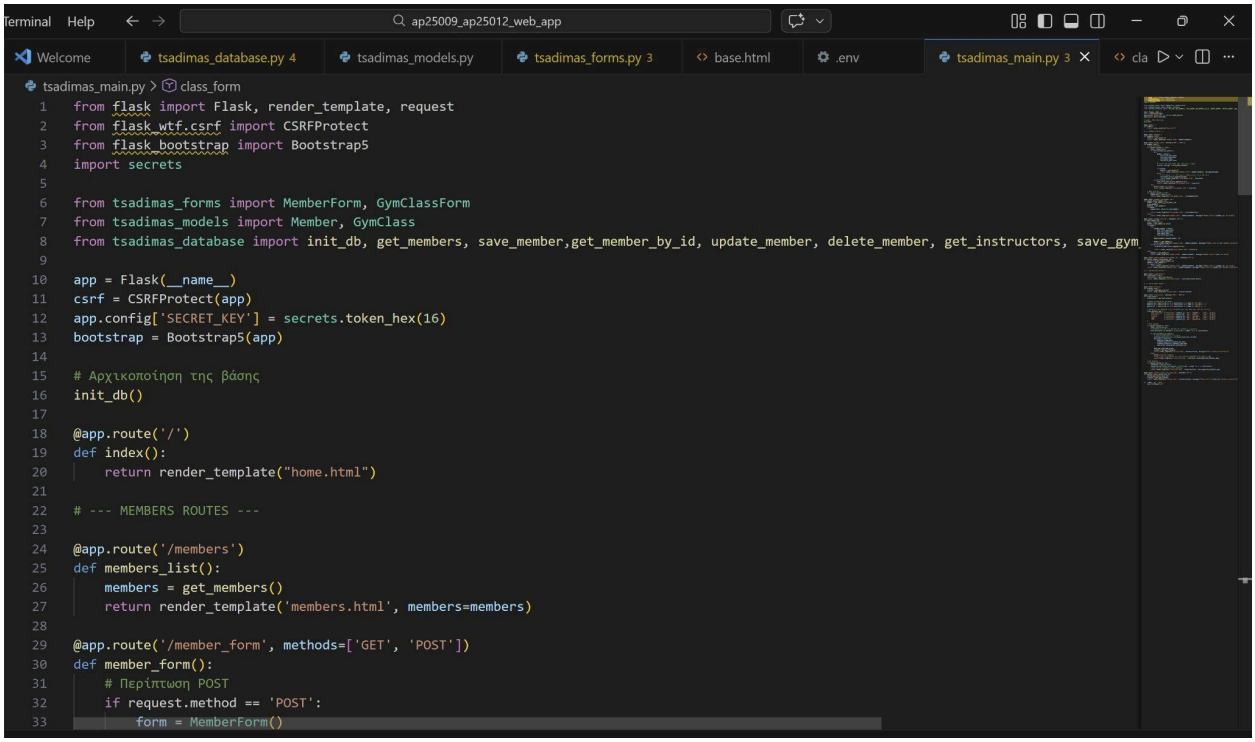

Class Form
127.0.0.1:5000/class_form
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
New Gym Class
Όνομα Μαθήματος
Crossfit
Ημέρα
Τρίτη
Ώρα
07:00 PM
Γυμναστής
Πέτρος
Δημιουργία

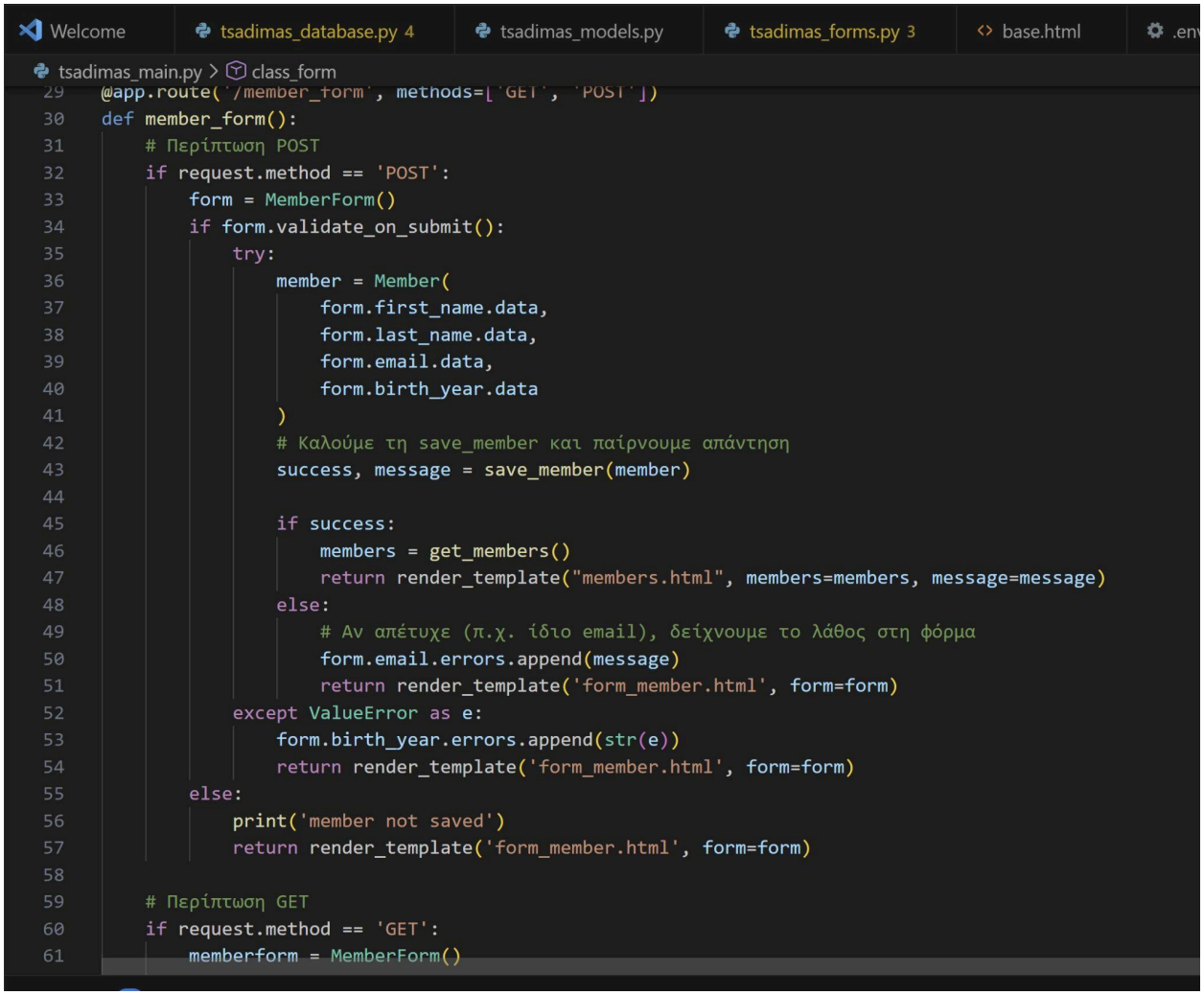


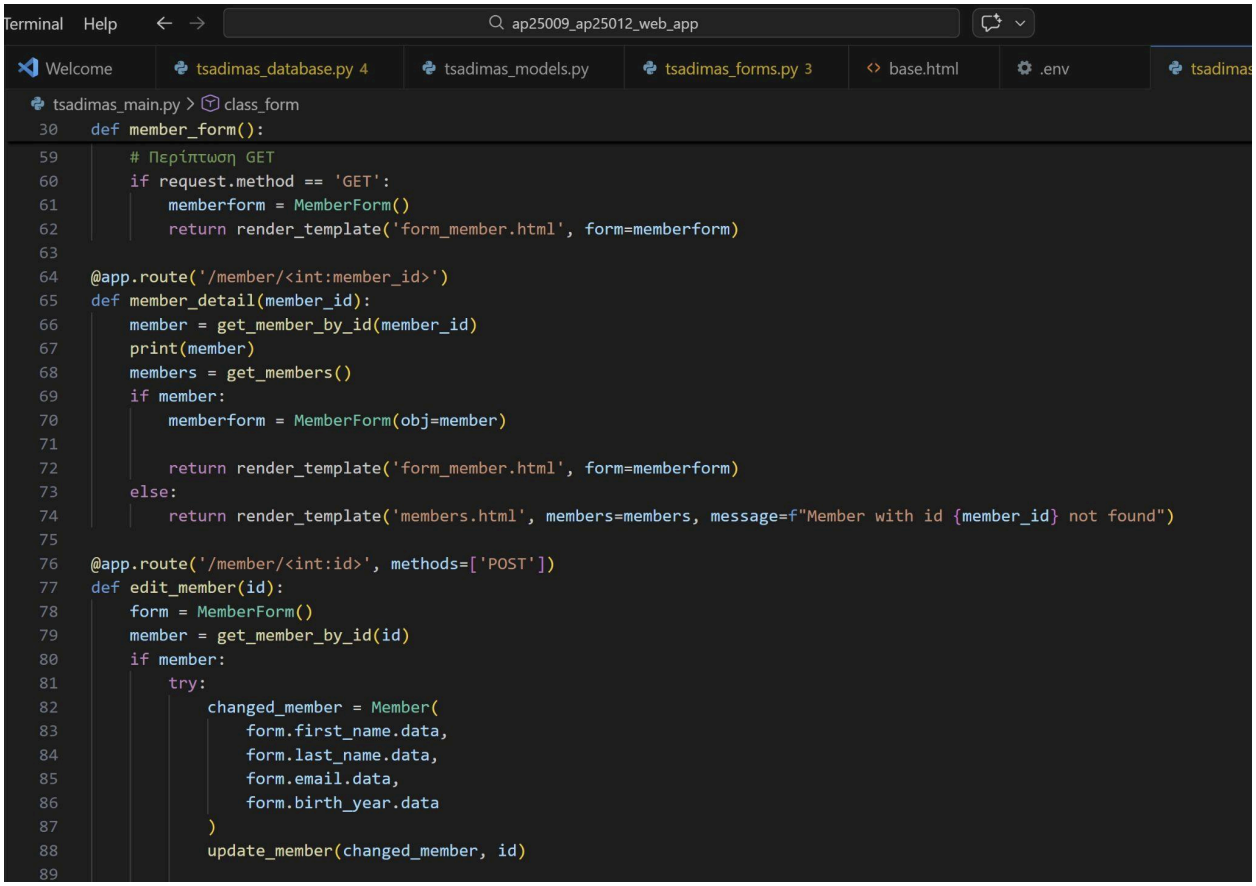
Δημιουργία αρχείου main

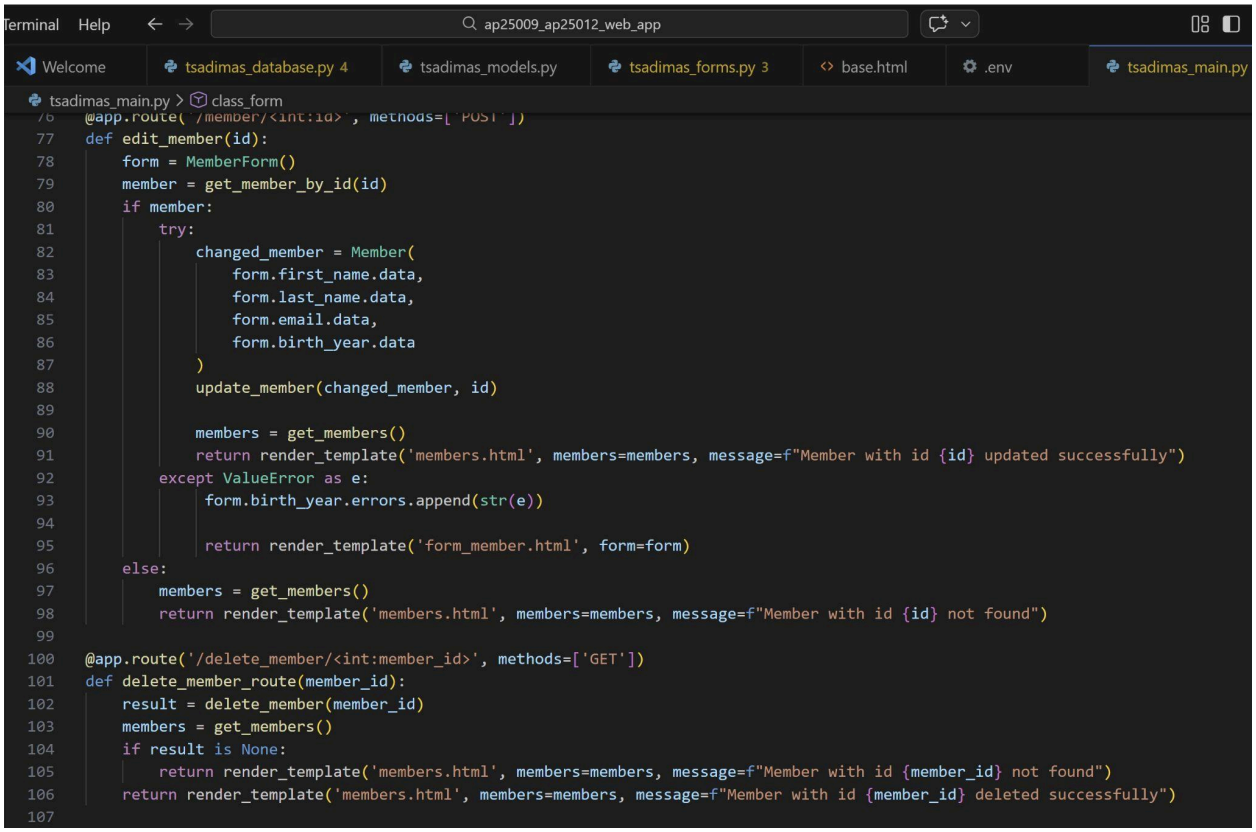
Το αρχείο `tsadimas_main.py` είναι αυτό που συνδέει όλα τα προηγούμενα αρχεία που αναλύσαμε. Λειτουργεί ως ο ελεγκτής (Controller) της εφαρμογής: όταν ο χρήστης ζητάει μια σελίδα, το `tsadimas_main.py` επικοινωνεί με τη βάση δεδομένων (μέσω των `tsadimas_ models` και του αρχείου `tsadimas_database`) και επιστρέφει τα κατάλληλα templates χρησιμοποιώντας τις φόρμες. Μέσω του μηχανισμού δρομολόγησης (Routing) και των εντολών `@app.route`, η Python αναγνωρίζει ποια σελίδα ζήτησε ο χρήστης και ενεργοποιεί την κατάλληλη συνάρτηση. Παράλληλα, ελέγχει αν η φόρμα είναι έγκυρη, διασφαλίζοντας ότι τηρούνται οι κανόνες που θέσαμε. Τέλος, εδώ γίνονται οι έλεγχοι για σφάλματα κατά την εισαγωγή στοιχείων. Αντί η εφαρμογή να τερματίσει απότομα, εμφανίζει φιλικά μηνύματα στον χρήστη, κάνοντας

την εμπειρία πιο ομαλή.



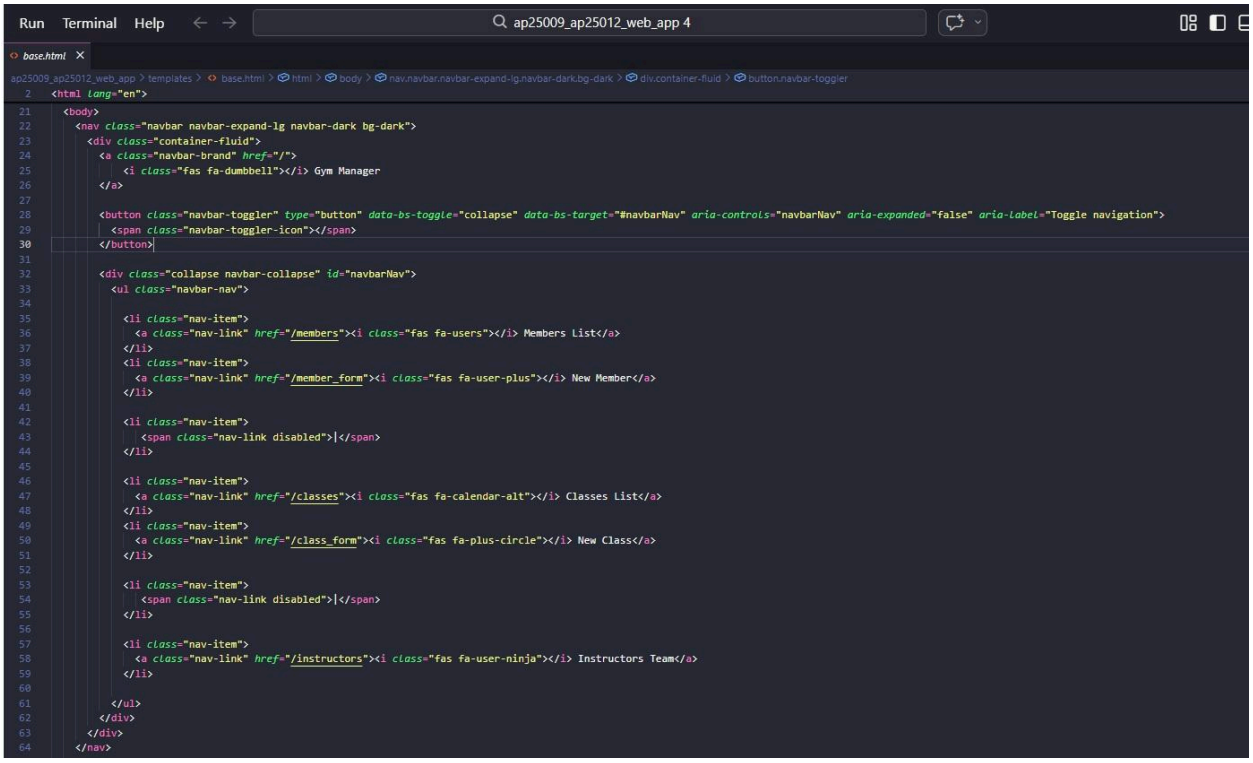

Welcome
tsadimas_database.py 4
tsadimas_models.py
tsadimas_forms.py 3
base.html
tsadimas_main.py > class_form
@app.route('/member_form', methods=['GET', 'POST'])
def member_form():
Περίπτωση POST
if request.method == 'POST':
form = MemberForm()
if form.validate_on_submit():
try:
member = Member(
form.first_name.data,
form.last_name.data,
form.email.data,
form.birth_year.data
)
Καλούμε τη save_member και παίρνουμε απάντηση
success, message = save_member(member)
if success:
members = get_members()
return render_template("members.html", members=members, message=message)
else:
Αν απέτυχε (π.χ. ίδιο email), δείχνουμε το λάθος στη φόρμα
form.email.errors.append(message)
return render_template('form_member.html', form=form)
except ValueError as e:
form.birth_year.errors.append(str(e))
return render_template('form_member.html', form=form)
else:
print('member not saved')
return render_template('form_member.html', form=form)
Περίπτωση GET
if request.method == 'GET':
memberform = MemberForm()


Terminal Help
ap25009_ap25012_web_app
Welcome
tsadimas_database.py 4
tsadimas_models.py
tsadimas_forms.py 3
base.html
.env
tsadimas_main.py > class_form
30 def member_form():
59 # Περίπτωση GET
60 if request.method == 'GET':
61 memberform = MemberForm()
62 return render_template('form_member.html', form=memberform)
63
64 @app.route('/member/<int:member_id>')
65 def member_detail(member_id):
66 member = get_member_by_id(member_id)
67 print(member)
68 members = get_members()
69 if member:
70 memberform = MemberForm(obj=member)
71
72 return render_template('form_member.html', form=memberform)
73 else:
74 return render_template('members.html', members=members, message=f"Member with id {member_id} not found")
75
76 @app.route('/member/<int:id>', methods=['POST'])
77 def edit_member(id):
78 form = MemberForm()
79 member = get_member_by_id(id)
80 if member:
81 try:
82 changed_member = Member(
83 form.first_name.data,
84 form.last_name.data,
85 form.email.data,
86 form.birth_year.data
87)
88 update_member(changed_member, id)
89

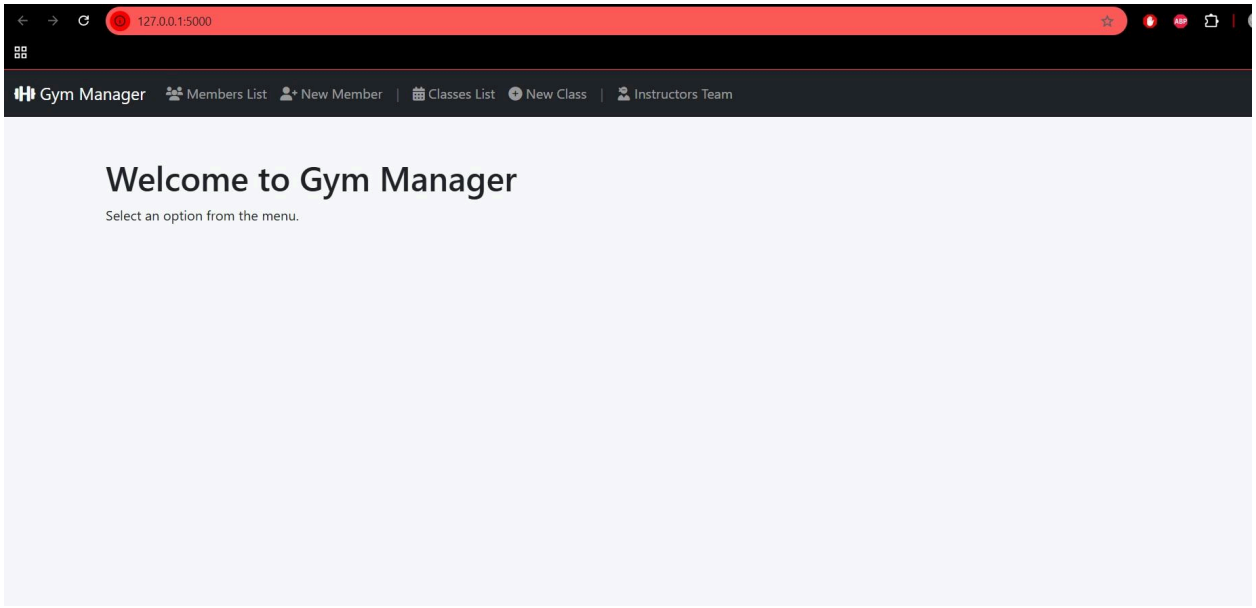


Εξήγηση των Templates αρχείων

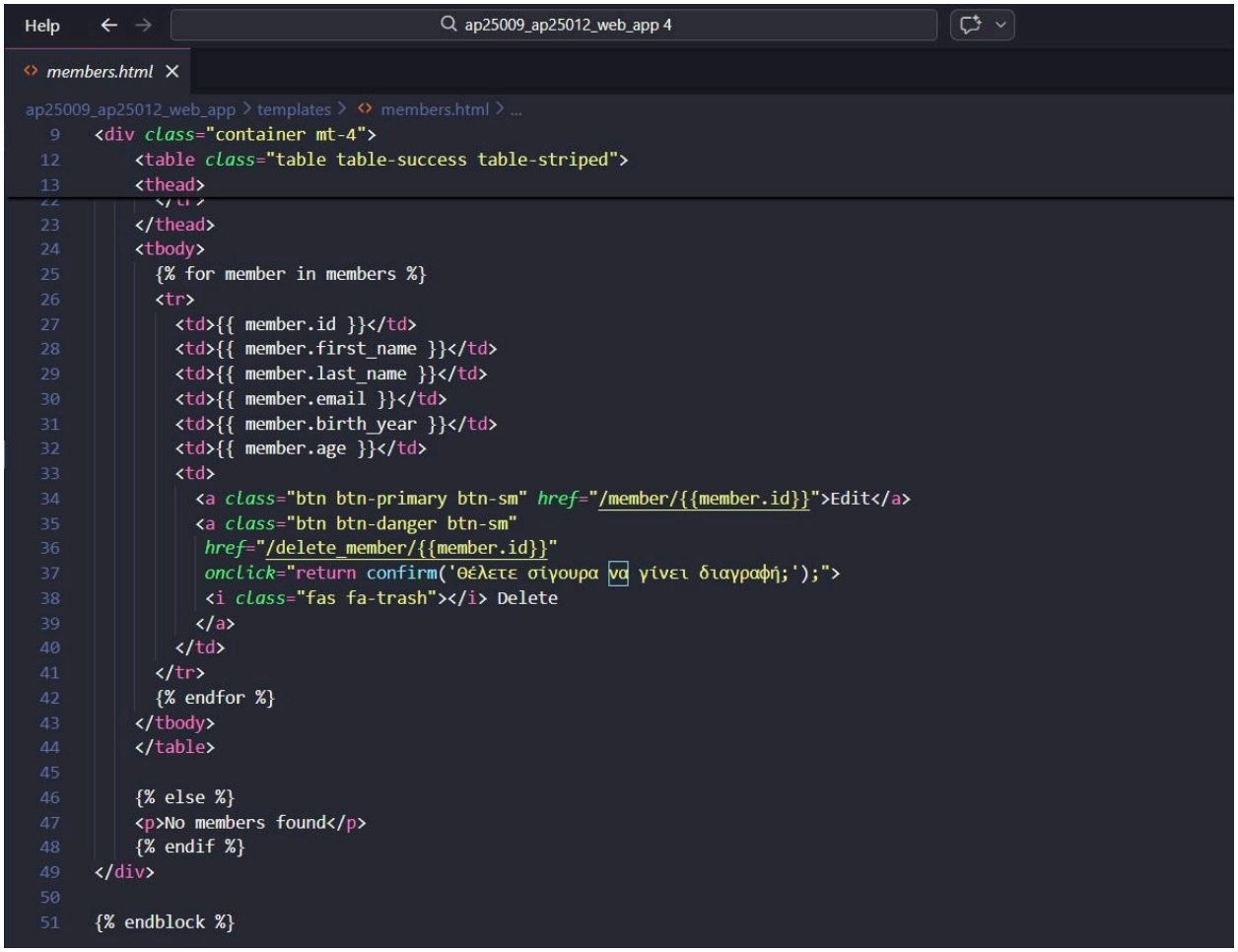
Το αρχείο `base.html` , είναι η "βάση" που θα στηριχθεί η εφαρμογή. Σε αυτό το αρχείο, έχουμε χρησιμοποιήσει το Bootstrap, για να μορφοποιήσουμε το σαιτ και το FontAwesome για να φτιάξουμε τα εικονίδια. Έτσι, όλες οι σελίδες έχουν έτοιμη την εμφάνιση χωρίς να χρειάζεται να προσθέτουμε κώδικα σε καθεμία ξεχωριστά. Επιπλέον, περιέχει την μπάρα του μενού (`<nav>`) και την οποια την καναμε σκουρόχρωμη (`bg-dark`) και τελος προσθέσαμε ένα χώρο που ελέγχει αν υπάρχει κάποιο νέο μήνυμα (`{% if message %}`). Αν, για παράδειγμα, αποθηκεύσουμε επιτυχώς ένα μέλος, εδώ θα εμφανιστεί το ενημερωτικό πλαίσιο. Χρησιμοποιήσαμε το Jinja2Blocks για να δημιουργήσαμε κενά σημεία (`{% block content %}` και `{% block title %}`) ώστε και οι υπόλοιπες σελίδες της εφαρμογής απλώς «κουμπώνουν» το δικό τους κείμενο μέσα σε αυτά.




```
            </ul>
          </div>
        </div>
      </nav>
      {% if message %}
      <div class="alert alert-warning alert-dismissible fade show" role="alert">
        <strong>{{ message }}</strong>
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
      </div>
      {% endif %}
      {% block content %}
      {% endblock %}
      {% block scripts %}
      {{ bootstrap.load_js() }}
      {% endblock %}
    </body>
  </html>
```



Τα αρχεια `members_html` και `class_html` εμφανίζουν τα δεδομένα που αντλούνται από τη βάση PostgreSQL, χρησιμοποιώντας το Jinja2.Εδω το σύστημα ελέγχει πρώτα αν υπάρχουν εγγραφές (`{% if ... %}`). Αν η βάση είναι άδεια, εμφανίζει το ενημερωτικό μήνυμα «No members found» αντί για έναν κενό πίνακα. Έπειτα χρησιμοποιούμε ένα βρόχο επανάληψης (`{% for ... %}`), ο οποίος δημιουργεί αυτόματα μια νέα γραμμή στον πίνακα για κάθε εγγραφή που υπάρχει στη βάση. Οπότε τώρα στη λίστα των μαθημάτων, το σύστημα εμφανίζει απευθείας το Όνομα του Γυμναστή (και όχι το ID του), κάνοντας την πληροφορία πιο εύκολη για τον χρήστη. Τέλος στα κουμπιά διαγραφής για τους members και για τα μαθήματα προσθέσαμε την εντολή επιβεβαίωσης (confirm). Έτσι, πριν διαγραφεί οριστικά μια εγγραφή, εμφανίζεται ένα παράθυρο που μας ρωτάει αν είμαστε σίγουροι για αυτό.


Help
ap25009_ap25012_web_app 4
members.html
ap25009_ap25012_web_app > templates > members.html > ...

<table class="table table-success table-striped">
<thead>
</thead>
<tbody>
{% for member in members %}
<tr>
<td>{{ member.id }}</td>
<td>{{ member.first_name }}</td>
<td>{{ member.last_name }}</td>
<td>{{ member.email }}</td>
<td>{{ member.birth_year }}</td>
<td>{{ member.age }}</td>
<td>
Edit
<a class="btn btn-danger btn-sm"
href="/delete_member/{{member.id}}"
onclick="return confirm('Θέλετε σίγουρα να γίνει διαγραφή;');">
 Delete

</td>
</tr>
{% endfor %}
</tbody>
</table>
{% else %}
No members found
{% endif %}

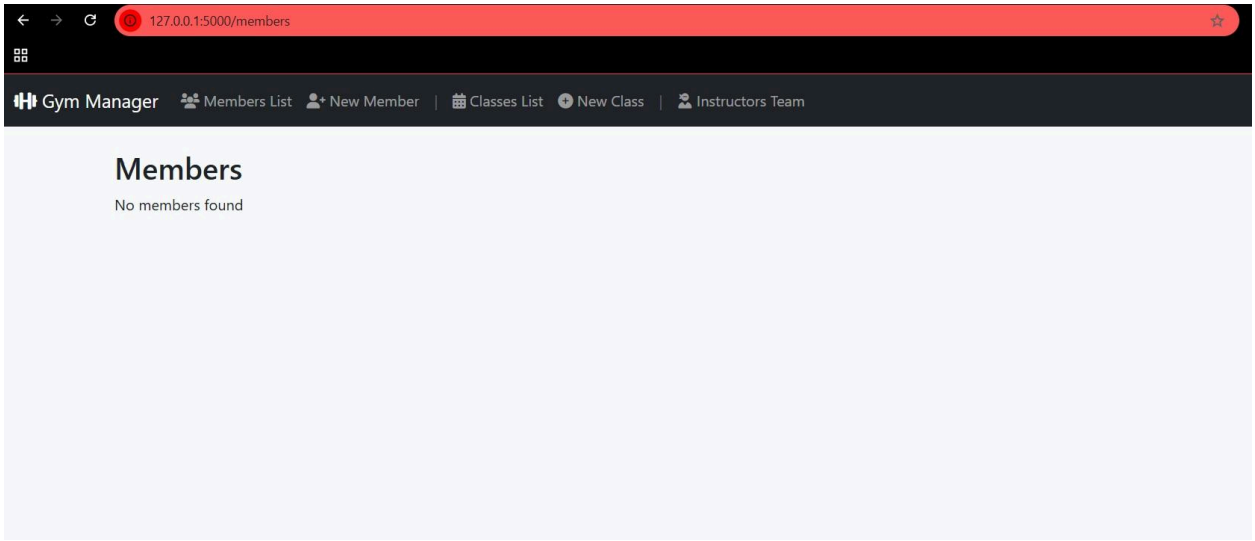
{% endblock %}

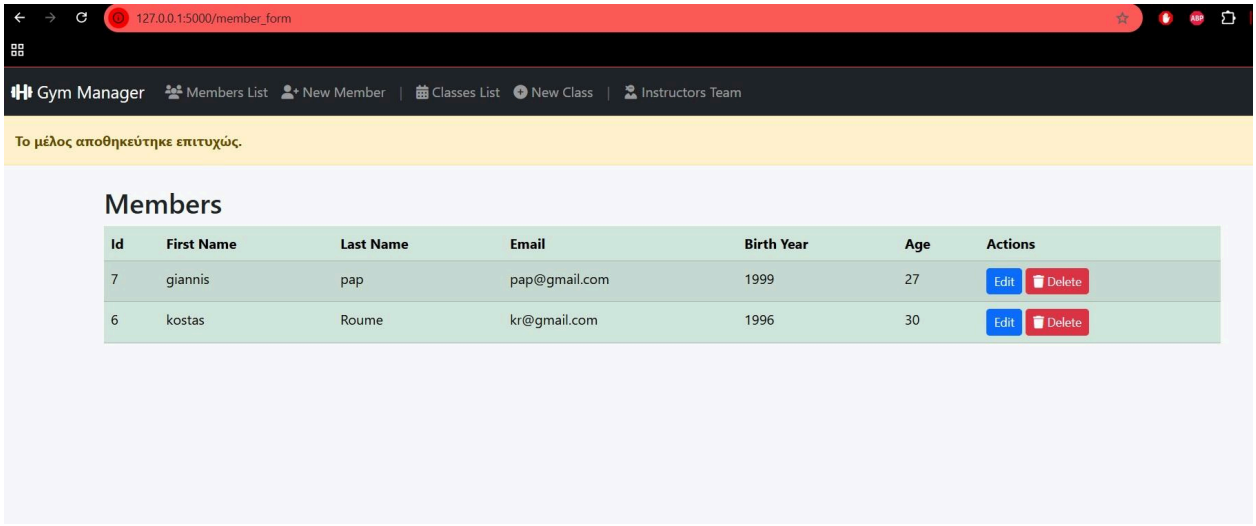
```
    </thead>
    <tbody>
      {% for class in classes %}
      <tr>
        <td>{{ class.id }}</td>
        <td>{{ class.name }}</td>
        <td>{{ class.schedule_day }}</td>
        <td>{{ class.schedule_time }}</td>
        <td>{{ class.instructor_name }}</td>
        <td>
         <a class="btn btn-danger btn-sm"
          href="/delete_cl
          onclick="return confirm('Θέλετε σίγουρα να γίνει διαγραφή;');">
          <i class="fas fa-trash"></i> Delete
         </a>
        </td>
      </tr>
      {% endfor %}
    </tbody>
    </table>

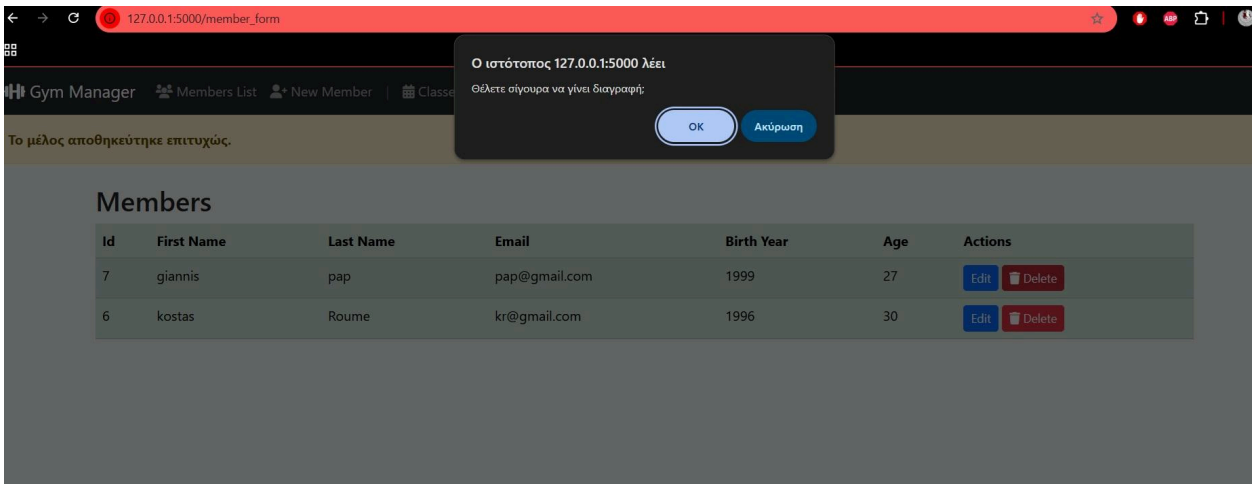
    {% else %}
    <p>No classes found</p>
    {% endif %}
</div>

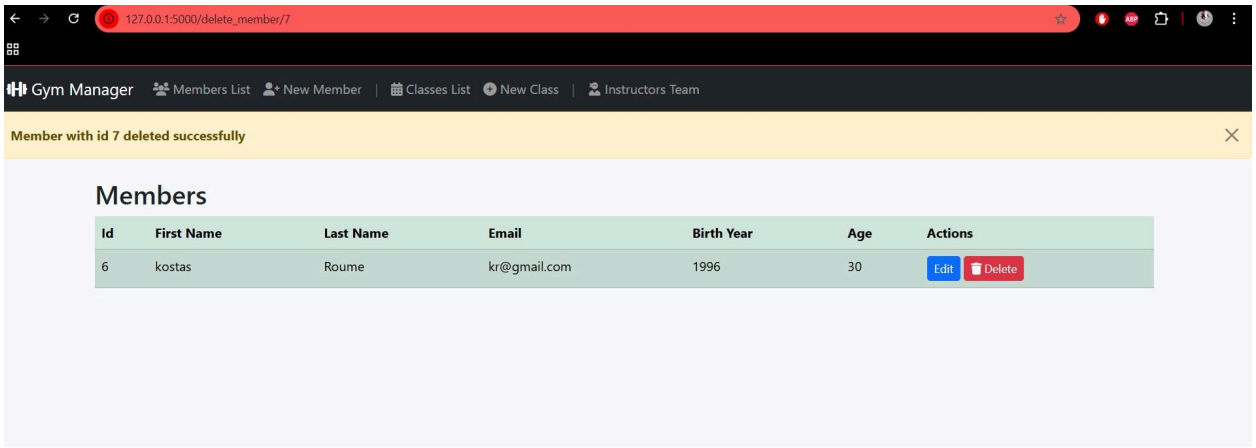
{% endblock %}
```

function confirm(message?: string): boolean




127.0.0.1:5000/member_form
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
Το μέλος αποθηκεύτηκε επιτυχώς.
Members
Id
First Name
Last Name
Email
Birth Year
Age
Actions
7
giannis
pap
pap@gmail.com
1999
27
Edit
Delete
6
kostas
Roume
kr@gmail.com
1996
30
Edit
Delete


127.0.0.1:5000/member_form
Ο ιστότοπος 127.0.0.1:5000 λέει
Θέλετε σίγουρα να γίνει διαγραφή;
OK
Ακύρωση
Gym Manager
Members List
New Member
Το μέλος αποθηκεύτηκε επιτυχώς.
Members
Id
First Name
Last Name
Email
Birth Year
Age
Actions
7
giannis
pap
pap@gmail.com
1999
27
Edit
Delete
6
kostas
Roume
kr@gmail.com
1996
30
Edit
Delete


127.0.0.1:5000/delete_member/7
Gym Manager
Members List
New Member
Classes List
New Class
Instructors Team
Member with id 7 deleted successfully
Members
Id
First Name
Last Name
Email
Birth Year
Age
Actions
6
kostas
Roume
kr@gmail.com
1996
30
Edit
Delete

Στο αρχείο `form_class.html` υλοποιείται η φόρμα καταχώρησης νέων μαθημάτων. Με τη χρήση του `render_form` αντί να γράφουμε χειροκίνητα HTML κώδικα για κάθε πεδίο, αξιοποιούμε τη συνάρτηση `render_form(form)` της βιβλιοθήκης `bootstrap-flask`. Με αυτό έχουμε δύο πλεονεκτήματα: αρχικά μετατρέπει αυτόματα τα πεδία της python (όπως `SelectField` για λίστες επιλογών και `TimeField` για την ώρα) σε έτοιμο κώδικα HTML, εφαρμόζοντας το στυλ του Bootstrap framework και στην συνέχεια αν ο χρήστης παραλείψει ένα υποχρεωτικό πεδίο, η φόρμα επαναφορτώνεται και εμφανίζει αυτόματα το σχετικό μήνυμα λάθους με κόκκινο χρώμα, χωρίς να απαιτείται επιπλέον κώδικας από εμάς.

Για να επιταχύνουμε τη διαδικασία εισαγωγής δεδομένων και να μειώσουμε τα λάθη, ενσωματώσαμε ένα σενάριο εντολών JavaScript στο κάτω μέρος της σελίδας. (Τα οποία δημιουργήθηκαν εξ'ολοκλήρου από AI). Η χρήση του οφείλεται στο γεγονός ότι θέλαμε να ενώσουμε τους γυμναστές με συγκεκριμένα μαθήματα, μέρες και ώρες γιατί αλλιώς ο κάθε γυμναστής μπορούσε να μπει σε όλα τα μαθήματα. Η σύνδεση έγινε μέσω της εντολής `const classMap = {{ class_map | tojson }};`

Terminal Help ap25009_ap25012_web_app 4

form_class.html 4

ap25009_ap25012_web_app > templates > form_class.html > ...

```
<div class="container mt-4">
 <h2>New Gym Class</h2>
 <div class="row">
   <div class="col-md-10 col-lg-8 mx-lg-auto mx-md-auto">
     {{ render_form(form, id="classForm") }}
   </div>
 </div>
</div>

<script>
  // 1. Παίρνουμε τα δεδομένα από την Python
  // Η εντολή tojson το μετατρέπει σε μορφή που καταλαβαίνει η Javascript
  const classMap = {{ class_map | tojson }};

  // 2. Βρίσκουμε τα κουτάκια (inputs) της φόρμας
  const classNameField = document.getElementById('name');           // Dropdown Μαθήματος
  const instructorField = document.getElementById('instructor_id'); // Dropdown Γυμναστή
  const dayField = document.getElementById('schedule_day');         // Dropdown Ημέρας
  const timeField = document.getElementById('schedule_time');       // Πεδίο Ώρας

  // 3. Όταν ο χρήστης αλλάξει το Μάθημα...
  if (classNameField) {
      classNameField.addEventListener('change', function() {
          const selectedClass = this.value; // Π.χ. 'Pilates'

          // Αν υπάρχει αυτό το μάθημα στον χάρτη μας
          if (classMap[selectedClass]) {
              const details = classMap[selectedClass];

              // Συμπλήρωσε τον Γυμναστή
              if (details.instructor && instructorField) {
                  instructorField.value = details.instructor;
              }
              // Συμπλήρωσε την Ημέρα
              if (details.day && dayField) {
                  dayField.value = details.day;
              }
              // Συμπλήρωσε την Ώρα
              if (details.time && timeField) {
                  timeField.value = details.time;
              }
          }
      });
  }
</script>
```

127.0.0.1:5000/class_form

Gym Manager | Members List | New Member | Classes List | New Class | Instructors Team

New Gym Class

Όνομα Μαθήματος

Crossfit

Ημέρα

Τρίτη

Ώρα

07:00 μμ

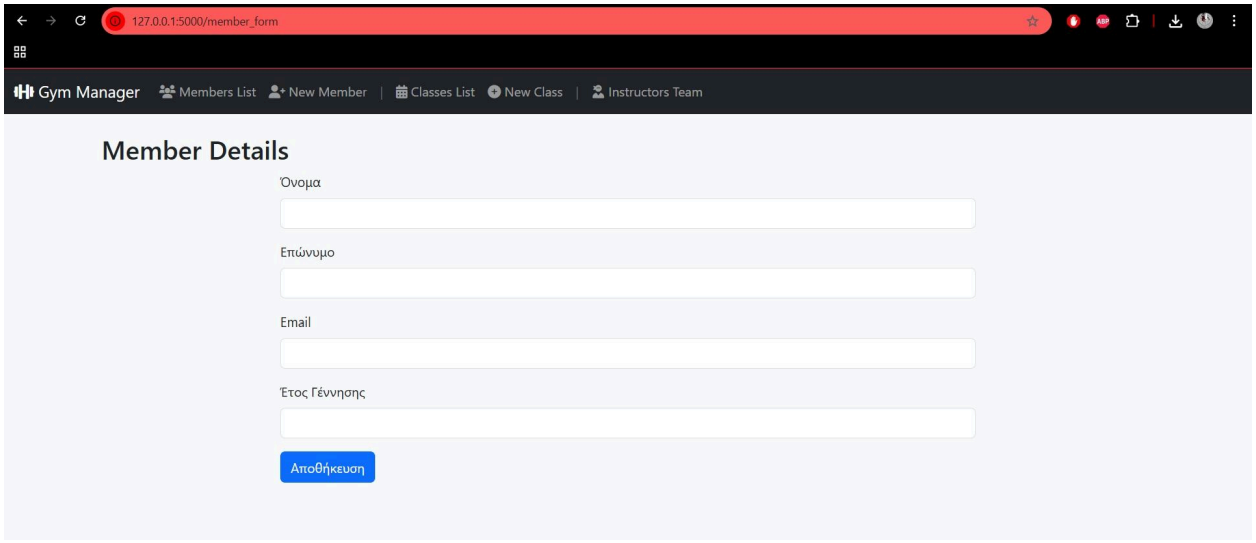
Γυμναστής

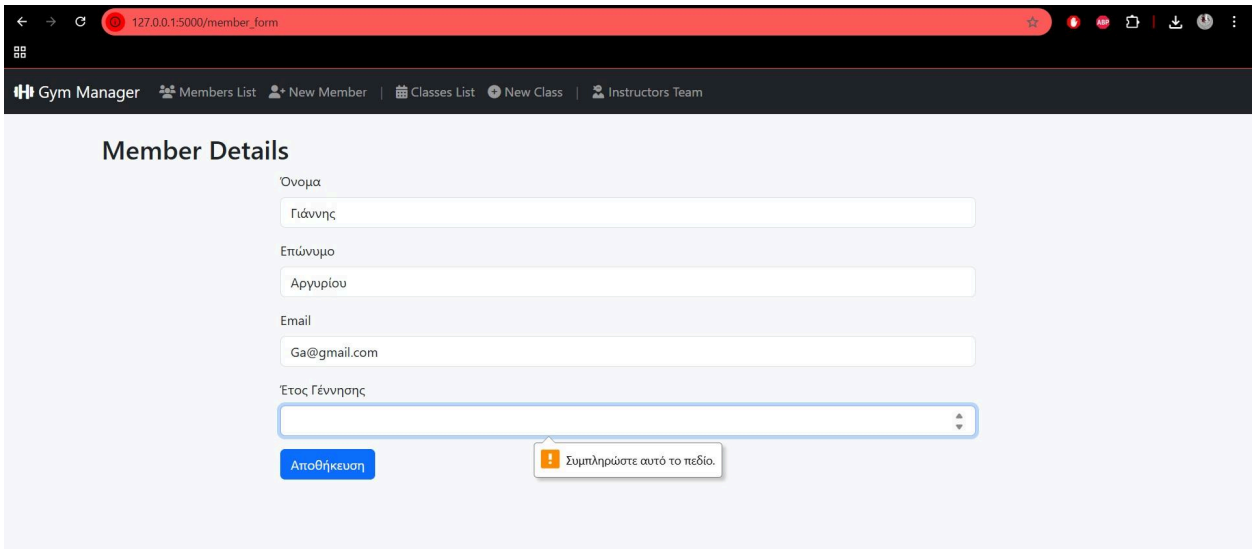
Πέτρος

Δημιουργία

Για την εισαγωγή νέων μελών, χρησιμοποιήθηκε το πρότυπο `form_member.html`. Η σχεδίαση εστιάζει στην απλότητα και την ευχρηστία. Προσθέσαμε το CSS Grid του Bootstrap (col-md-10 col-lg-8 mx-auto) για να κεντράρει τη φόρμα στη σελίδα, προσφέροντας βέλτιστη αναγνωσιμότητα. Τέλος με την ενσωμάτωση του `Flask-WTF` εξασφαλίσαμε ότι όλα τα πεδία (Όνομα, Email, Έτος Γέννησης) ελέγχονται για την εγκυρότητά τους και ότι είναι συμπληρωμένα πριν την υποβολή.

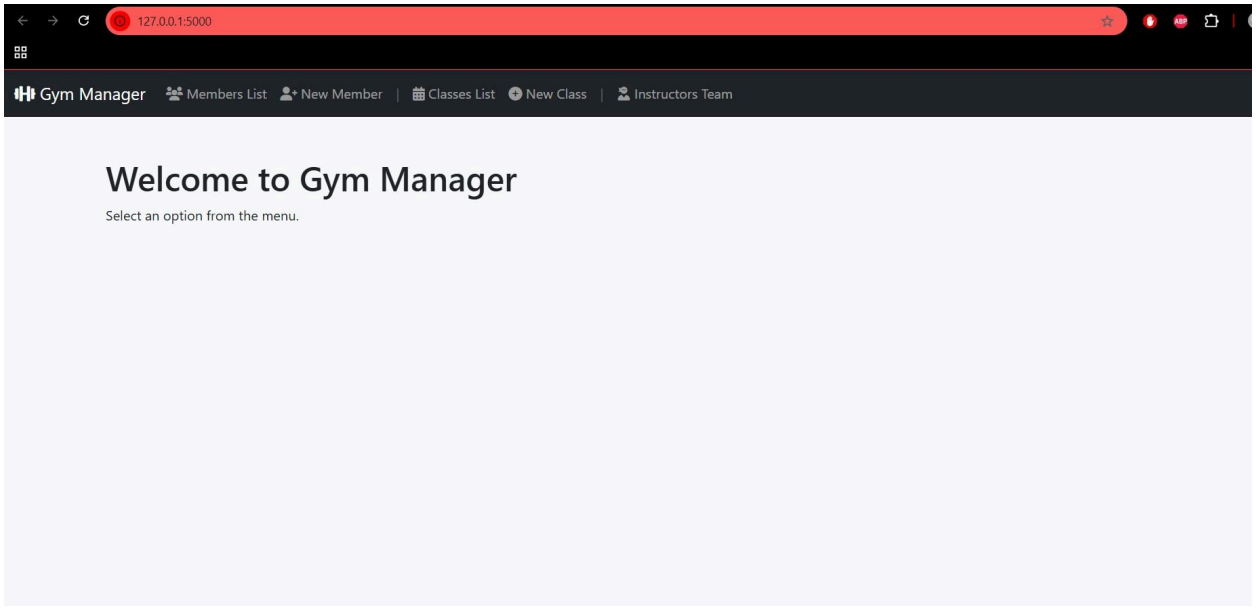






Το αρχείο `home.html` είναι το πιο απλό κομμάτι “κώδικα” της εφαρμογής. Ξεκινάει με την εντολή `extends`, η οποία συνδέει το αρχείο με το `base.html`. Στο εσωτερικό του, ορίζει τον τίτλο της καρτέλας ως **'Home page!'** και στο κυρίως μέρος εμφανίζει την επικεφαλίδα **'Welcome to Gym Manager'** μαζί με την οδηγία **'Select an option from the menu'**, τοποθετημένα μέσα σε ένα πλαίσιο (`div`) για σωστή στοίχιση.





Το αρχείο `instructors.html` περιέχει τον κώδικα για την παρουσίαση της λίστας των γυμναστών. Ξεχωρίζει αισθητικά καθώς ορίζει έναν πίνακα με σκούρο φόντο (`table-dark`), σε αντίθεση με τις υπόλοιπες λίστες της εφαρμογής. Επιπλέον, περιλαμβάνει ένα ενημερωτικό πλαίσιο (`alert-info`) με το μήνυμα **'Εδώ βλέπετε την ομάδα των γυμναστών μας'**, ενώ τα δεδομένα (ID και Όνομα) αντλούνται από τη βάση δεδομένων. Είναι ουσιαστικά τα ονόματα που καταχωρήσαμε στην βάση.

Terminal Help ap25009_ap25012_web_app 4

instructors.html

ap25009_ap25012_web_app > templates > instructors.html > ...

```
{% extends "base.html" %}

{% block title %}
Instructors List
{% endblock %}

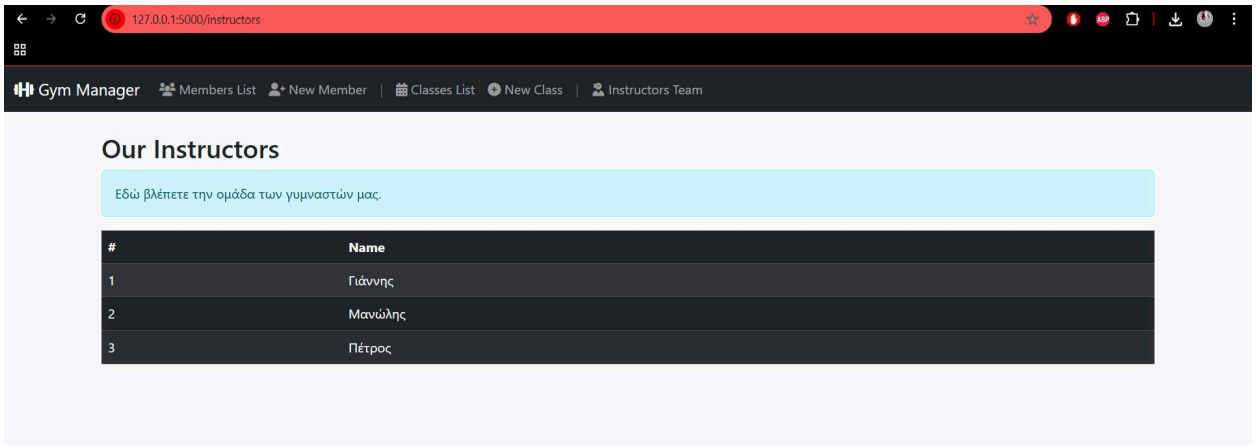
{% block content %}

<div class="container mt-4">
    <h2>Our Instructors</h2>
    <div class="alert alert-info" role="alert">
        Εδώ βλέπετε την ομάδα των γυμναστών μας.
    </div>

    {% if instructors | length > 0 %}
    <table class="table table-dark table-striped table-hover">
    <thead>
      <tr>
        <th scope="col">#</th>
        <th scope="col">Name</th>
        </tr>
    </thead>
    <tbody>
      {% for instructor in instructors %}
      <tr>
        <td>{{ instructor.id }}</td>
        <td>{{ instructor.name }}</td>
        </tr>
      {% endfor %}
    </tbody>
    </table>

    {% else %}
    <p>No instructors found.</p>
    {% endif %}
</div>

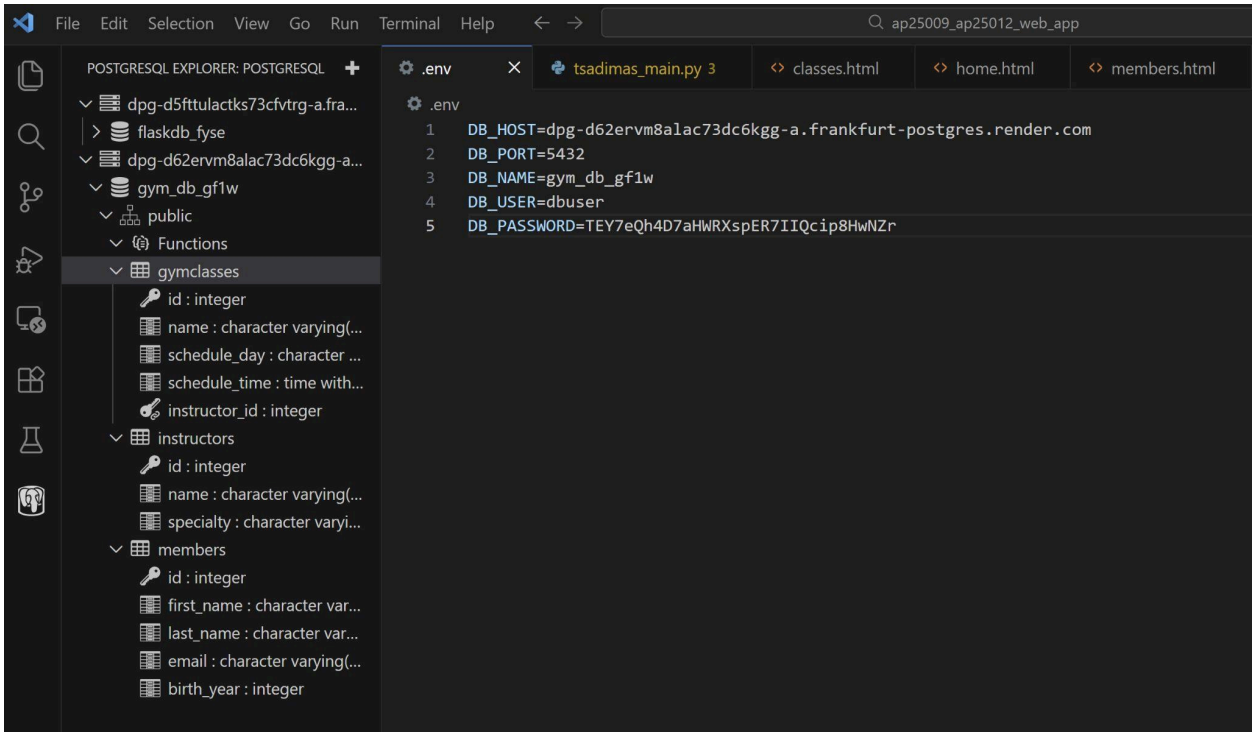
{% endblock %}
```



.

Τρόπος Εκτέλεσης

Για τις ανάγκες της εφαρμογής, δημιουργήσαμε μια απομακρυσμένη βάση δεδομένων PostgreSQL φιλοξενούμενη στην πλατφόρμα Render. Η σύνδεση με τον Server της βάσης επιτυγχάνεται μέσω του αρχείου .env όπου σε αυτό χρησιμοποιούμε μεταβλητές για να αποθηκεύσουμε τα στοιχεία για την σύνδεση της όπως φαίνεται και στον φάκελο.

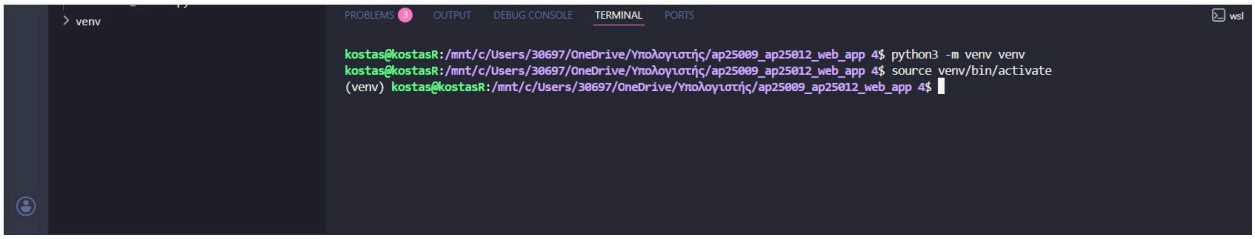


Αφού ολοκληρώσαμε την σύνδεση και την δημιουργία της βάσης μας ξεκινήσαμε την υλοποίηση της εφαρμογής όπως περιγράψαμε αναλυτικά τα βήματα πιο πάνω.

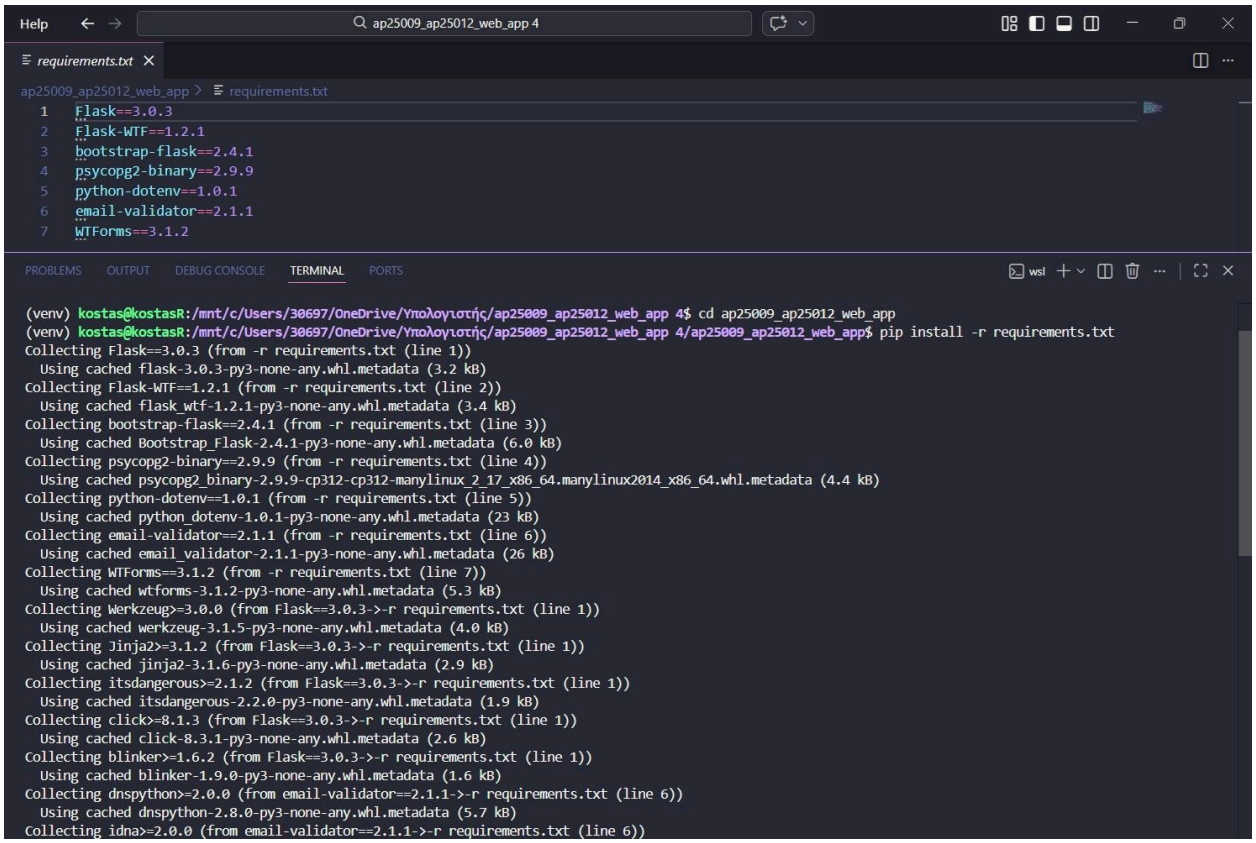
Για να τρέξουμε την εφαρμογή και να δούμε ότι “δουλεύει” αρχικά φτιάξαμε ένα εικονικό περιβάλλον.

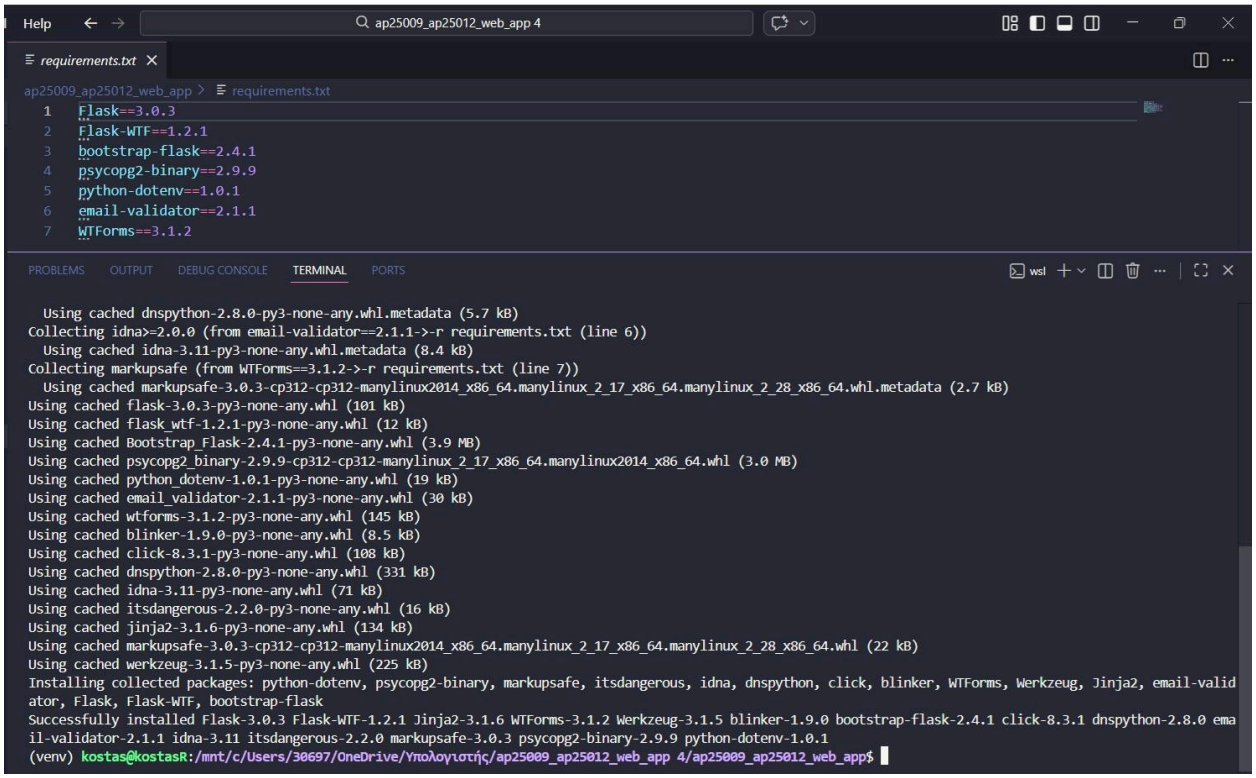


Στην συνέχεια το ενεργοποιήσαμε.



Έπειτα δημιουργήσαμε ένα αρχείο txt που το ονομάσαμε requirements στο οποίο προσθέσαμε όποια βιβλιοθήκη χρειαστήκαμε για να τρέξει η εφαρμογη.





Τέλος μέσω του terminal τρέξαμε το αρχείο `tsadimas_main.py`. Όπου εμφανίζεται το url της εφαρμογής.



Με δυσκόλεψε

Πρώτον, μας δυσκόλεψε η σύνδεση της βάσης δεδομένων στο Render. Παρόλο που βρήκαμε όλα τα απαραίτητα στοιχεία (host, port, κωδικούς) μέσω του PSQL Command, όταν προσπαθούσαμε να τα εισάγουμε στο VS Code για να συνδέσουμε τη βάση, μας εμφάνιζε συνέχεια σφάλμα για "time limit". Αυτό μας καθυστέρησε αρκετά μέχρι να καταφέρουμε να κάνουμε τη σύνδεση.

Δεύτερον, στον κώδικα, το δύσκολο ήταν να καταλάβουμε πώς θα εμφανίσουμε το όνομα του γυμναστή και όχι απλά το ID του. Σε αυτό το σημείο χρειάστηκε να χρησιμοποιήσουμε την εντολή SQL JOIN για να λύσουμε αυτο το πρόβλημα. Μας ήταν αδύνατον να σκεφτούμε πως θα τραβήξουμε τα στοιχεία του μαθήματος, αλλά ταυτόχρονα και να "ψάξουμε" στον πίνακα των γυμναστών για να βρούμε ποιο όνομα αντιστοιχεί σε αυτόν τον κωδικό. Οπότε χρησιμοποιήσαμε την βοήθεια του AI.

Τρίτον μας δυσκόλεψε και πειραματιστήκαμε στα templates της html, τι πρέπει να πειράξουμε (να προσθέσουμε ή να αφαιρέσουμε) σε αυτά από το bootstrap κλπ για να κάνουμε την εφαρμογή μας πιο εύκολη και ελκυστική στο χρήστη.

Τέλος, το κομμάτι το οποίο αποφασίσαμε να αλλάξουμε ήταν το `form_class.html` και ένα κομμάτι του `tsadimas_main.py`, παρόλο που είχαμε γράψει τον κώδικα. Στην αρχή στη σελίδα μας θα μπορούσε κάποιος να επιλέξει χειροκίνητα το μάθημα, την ημέρα, την ώρα, ακόμα και τον γυμναστή. Αυτό ήταν κάτι που πρακτικά δεν είναι ρεαλιστικό. Οπότε αρχίσαμε να ψάχνουμε έναν τρόπο που θα μπορούσαμε να συνδέσουμε τα μαθήματα με συγκεκριμένες ώρες, μέρες και γυμναστές. Χωρίς να βρίσκουμε κάποια λύση, ρωτήσαμε το AI και μας έδωσε έναν απλό κομμάτι κώδικα JS το οποίο έδινε λύση στο πρόβλημά μας και έκανε το web app μας πιο χρηστικό.

Βιβλιογραφία

Bootstrap Team. (n.d.). *Bootstrap 5.0 docs*
https://getbootstrap.com/docs/5.0/

Google. (2026). *Gemini* (Feb 12 version) [Large language model].
https://gemini.google.com/

Render. (n.d.). *Render documentation*.
https://render.com/docs

Διάλεξη-8-08-01-2026.mp4
https://drive.google.com/drive/u/1/home

Python web app presentation
https://docs.google.com/presentation/d/1G35uOg6a17cnXzL_ZYH87mPmdS4EF_PyBVjxdWNhuig/edit?slide=id.p#slide=id.p

Ανάπτυξη web εφαρμογών με python Flask framework
https://docs.google.com/document/d/1fFoUugNZLuE3Wp-myo5N0OhN25XaL017MAYWwMYLVQo/edit?tab=t.0#heading=h.agz5nyq466wv

Το αρχείο lec8_a.zip ήταν η βάση πάνω στην οποία στηριχτήκαμε για να υλοποιήσουμε την εργασία μας,

Συνοπτικός Πίνακας

2η Εργασία		
Λειτουργία	Υλοποιήθηκε **(ΝΑΙ/ΟΧΙ/ΜΕΡΙΚΩΣ)**	**Συνοπτικές Παρατηρήσεις**
Υλοποίηση σε Flask Framework	ΝΑΙ	Η εφαρμογή βασίζεται στη βιβλιοθήκη Flask
Σύνδεση με Βάση (PostgreSQL στο Render)	ΝΑΙ	Χρήση βιβλιοθήκης psycopg2 και σύνδεση με host στο Render.com.
Χρήση αρχείου παραμετροποίησης (.env)	ΝΑΙ	Οι κωδικοί της βάσης (DB_HOST, DB_PASS) είναι κρυμμένοι στο .env.
Χρήση Jinja2 template engine	ΝΑΙ	Χρήση {{ variable }} και {% for %} στα αρχεία HTML για εμφάνιση δεδομένων.
Αρχική σελίδα με βασικές πληροφορίες	ΝΑΙ	Το αρχείο home.html με το μήνυμα "Welcome to Gym Manager".
Σταθερό Menu επιλογών	ΝΑΙ	Η μπάρα πλοήγησης (Navbar) στο base.html που εμφανίζεται σε όλες τις σελίδες.
Λίστα αντικειμένων από 3 πίνακες	ΝΑΙ	1. Λίστα Μελών (members table) 2. Λίστα Μαθημάτων (gymclasses table) 3. Λίστα Γυμναστών (instructors table).
Καταχώρηση σε τουλάχιστον 2 πίνακες	ΝΑΙ	1. Εγγραφή νέου Μέλους (save_member) 2. Δημιουργία νέου Μαθήματος (save_gym_class).
2 σελίδες με ενδιαφέροντα Queries	ΝΑΙ	1. SQL JOIN: Στα Μαθήματα (get_gym_classes) για εμφάνιση ονόματος γυμναστή αντί ID. 2. ORDER BY: Ταξινόμηση μελών από το πιο πρόσφατο (ORDER BY id DESC).
Προσαρμογή / Επιπλέον Λειτουργικότητα	ΝΑΙ	1. Δυναμική Φόρμα: Αυτόματη συμπλήρωση & κλείδωμα πεδίων με JavaScript. 2. Validators: Έλεγχος ηλικίας (10-90) και email.